Universidad del Valle de Guatemala
Facultad de Ingeniería
Departamento de Ciencias de la Computación
CC3069 Computación Paralela y Distribuida
Ing. Juan Luis García

Proyecto 2
Fuerza bruta sobre DES con Open MPI

Esteban Cárcamo	23016
Nicolás Concuá	23197
Diego López	23747

Repositorio: https://github.com/nicoCT04/Proyecto2-Paralela

Guatemala, 14 de octubre de 2026

Índice

Introducción **4**
Objetivos 4
Programas desarrollados 4
Estructura del informe 5

Parte A.1 a A.3: DES y la búsqueda secuencial **6**
A.1 ¿Qué es DES? 6
A.2 Diagrama de flujo 8
A.3 Búsqueda secuencial con distintas longitudes de llave 9

Parte A.4 a A.6: Rutinas de la base y primitivas MPI **10**
A.4 Por qué no compila rpc/des_crypt.h 10
A.5 Las rutinas de la base 10
A.6 Primitivas MPI del aviso de llave 12

Parte B.1 y B.2: Pruebas del cifrado y del naive **15**
B.1 Cifrar y descifrar desde un .txt 15
B.2 Naive con 4 procesos contra el secuencial 15

Enfoques de paralelización **20**
Esqueleto común 20
Naive: bloques contiguos 21
Intercalado 21
Bloque-cíclico 22
Maestro-trabajador 22
Híbrido MPI + OpenMP 25
Tabla teórica 26
Consistencia frente a promedio 27

Análisis de speedup y del comportamiento según la llave **28**
Lo teórico frente a lo medido 28
Por qué el naive es inconsistente 29
Por qué las alternativas son consistentes 30
El superlineal es aparente 30
Fugas de eficiencia 31
Barrido de procesos, Amdahl y efectividad 31
El híbrido frente a la versión sin OpenMP 33

De 28 a 56 bits 34

Conclusiones y recomendaciones 35

Conclusiones 35

Retos y cómo se resolvieron 36

Aprendizajes 36

Recomendaciones 37

Referencias 38

Anexo 1: Catálogo de funciones y librerías 40

des_utils.c: DES sobre OpenSSL 40

options.c: Parámetros 42

problem.c: Problema y reporte 43

cifrar.c: Parte B.1 45

secuencial.c: Parte A.3 46

search_mpi.c: Módulo común MPI 47

Programas MPI 51

hibrido.c: MPI + OpenMP 53

scripts/analizar.py: Análisis de la campaña 54

Librerías 56

Anexo 2: Bitácora de pruebas y speedups 59

Máquina y software 59

Procedimiento 59

Verificación 60

Pruebas B.1 y B.2 60

Campaña: tiempos (mediana de 5, en segundos) 60

Speedups (S = Ts / Tp) 61

Barrido de procesos (llave media) 61

Capturas 61

Introducción

El *Data Encryption Standard* (DES) protege un texto con una llave de 56 bits. Hoy esa llave es demasiado corta: probar las $2^{56} \approx 7.2 \cdot 10^{16}$ combinaciones está al alcance de una máquina dedicada (Electronic Frontier Foundation [EFF], 1998). Por eso el Instituto Nacional de Estándares y Tecnología de Estados Unidos retiró el estándar en 2005 (National Institute of Standards and Technology [NIST], 2005).

Ese ataque, la fuerza bruta, es un buen problema para estudiar la programación paralela. Cada llave se prueba de forma independiente, así que el trabajo se reparte casi sin comunicación. Sin embargo, la forma de repartirlo decide si el speedup es excelente o nulo.

En este proyecto se cifra un texto con DES y una llave privada. Después se recupera la llave probando combinaciones hasta que el texto descifrado contiene una frase clave conocida, «es una prueba de». La búsqueda está escrita en C, con OpenSSL para DES y Open MPI para repartir el trabajo entre procesos (Message Passing Interface Forum [MPI Forum], 2023). Como extra, OpenMP reparte el trabajo de cada proceso entre hilos (OpenMP Architecture Review Board [OpenMP ARB], 2021).

Objetivos

El objetivo general es implementar y modificar un programa paralelo con MPI en C que descifre por fuerza bruta un texto cifrado con DES. Primero se resuelve de forma secuencial y después con el reparto naive del programa base del curso.

Sobre esa base se proponen enfoques alternativos de reparto. Cada uno se mide con las métricas del curso: speedup, eficiencia, costo y efectividad, junto con las leyes de Amdahl (1967) y Gustafson (1988).

El último objetivo es explicar por qué el naive se comporta de forma inconsistente según la llave. Con esa explicación se demuestra que el proyecto no es superlineal: los speedups mayores que p que aparecen son un efecto de búsqueda.

Programas desarrollados

La tabla resume los siete programas del proyecto y el capítulo donde se discute cada uno.

Programa	Qué hace	Capítulo
`cifrar`	Cifra y descifra un `.txt` con una llave (Parte B.1)	Parte A y pruebas B

Programa	Qué hace	Capítulo
`secuencial`	Fuerza bruta en un solo proceso (Parte A.3); da el T_s	Parte A
`naive`	Bloques contiguos, uno por proceso (Parte B.2)	Pruebas B y enfoques
`intercalado`	El proceso r prueba r, r + p, r + 2p, …	Enfoques
`ciclico`	Chunks de 2^c llaves en round-robin	Enfoques
`maestro_trabajador`	El proceso 0 reparte chunks bajo demanda	Enfoques
`hibrido`	Intercalado con p procesos MPI y T hilos de OpenMP (extra)	Enfoques

Todos comparten el mismo arranque, el mismo aviso de llave y el mismo reporte, que viven en el módulo `search_mpi`. Todos escriben además una línea CSV con el mismo formato. Tres scripts completan el trabajo: `verificar.sh` comprueba que cada programa halla la llave real, `campana.sh` corre las mediciones con 5 repeticiones por caso y `analizar.py` genera las tablas y gráficas a partir de la mediana.

De los extras opcionales se abordaron dos: tres enfoques alternativos y OpenMP. El clúster en red local se descartó. Exigía las tres máquinas a la vez, con la misma versión de Open MPI y el mismo sistema operativo, y mezclar macOS con Linux lo volvía frágil para el tiempo disponible.

Estructura del informe

Los antecedentes corresponden a la Parte A. Allí se describe el algoritmo DES con su diagrama de flujo, el tiempo de la búsqueda secuencial con distintas longitudes de llave, las rutinas de la base (`encrypt_des`, `decrypt_des`, `try_key`, `memcpy` y `strstr`) y las primitivas MPI del aviso de llave.

El desarrollo corresponde a la Parte B. Incluye las pruebas B.1 y B.2 del naive con las llaves del enunciado, los enfoques alternativos con su análisis teórico y el enfoque híbrido.

Los resultados están en el análisis de speedup. Ese capítulo compara los programas por llave, presenta el barrido de procesos, aplica Amdahl y Gustafson y discute el superlineal. Siguen las conclusiones y recomendaciones, las referencias y dos anexos: el catálogo de funciones (Anexo 1) y la bitácora de pruebas (Anexo 2).

Parte A.1 a A.3: DES y la búsqueda secuencial

A.1 ¿Qué es DES?

El *Data Encryption Standard* (DES) es un algoritmo de cifrado simétrico por bloques. La misma llave sirve para cifrar y para descifrar. Se publicó como estándar federal de Estados Unidos en 1977, a partir del cifrado *Lucifer* de IBM, y su última revisión es la FIPS PUB 46-3 de 1999 (NIST, 1999). El NIST lo retiró en 2005 porque su llave es demasiado corta para la capacidad de cómputo actual (NIST, 2005).

Característica	Valor
Tamaño de bloque	64 bits (8 bytes)
Tamaño de llave	64 bits, de los cuales 8 son de paridad: 56 bits efectivos
Espacio de llaves	$2^{56} = 72\,057\,594\,037\,927\,936 \approx 7.2 \cdot 10^{16}$
Estructura	Red de Feistel de 16 rondas
Subllaves	16 subllaves de 48 bits, una por ronda

El último bit de cada uno de los 8 bytes de la llave es de paridad impar y no interviene en el cifrado. Por eso, aunque la llave ocupa 64 bits, la fuerza bruta solo tiene que recorrer 2^{56} combinaciones.

Cifrado de un bloque

El bloque de 64 bits pasa primero por una permutación inicial (IP), que reordena sus bits según una tabla fija. El resultado se parte en dos mitades de 32 bits, L_0 a la izquierda y R_0 a la derecha.

Luego vienen 16 rondas de Feistel. En la ronda i, la nueva mitad izquierda es la derecha anterior, $L_i = R_{i-1}$, y la nueva derecha es $R_i = L_{i-1} \oplus f(R_{i-1}, K_i)$. Solo la mitad derecha pasa por la función f; la izquierda solo se combina con un XOR.

Tras la ronda 16 se intercambian las mitades y el bloque queda $R_{16} \,\|\, L_{16}$. Por último, la permutación final IP^{-1}, inversa de la inicial, entrega el bloque cifrado de 64 bits.

La función f

La función f(R, K) recibe los 32 bits de la mitad derecha y los 48 bits de la subllave de la ronda (NIST, 1999; Stallings, 2020). Primero, la expansión E lleva la mitad derecha de 32 a 48 bits duplicando 16 de ellos. Después se combina con la subllave mediante un XOR.

Los 48 bits resultantes se parten en 8 grupos de 6, y cada grupo entra a una caja S. Cada caja convierte 6 bits en 4 con una tabla: los bits de los extremos eligen la fila y los 4 centrales, la columna. Las cajas S son la única parte no lineal de DES y la fuente de su seguridad. Los 32 bits que salen se reordenan con la permutación P.

Generación de subllaves

La permutación PC-1 toma la llave de 64 bits, descarta los 8 bits de paridad y reordena los 56 restantes. Esos 56 bits se dividen en dos mitades de 28, C_0 y D_0.

En cada ronda, C y D rotan a la izquierda una posición en las rondas 1, 2, 9 y 16, y dos posiciones en las demás. En total suman 28, así que al final vuelven a su estado inicial. De los 56 bits de $C_i \,\|\, D_i$, la permutación PC-2 elige 48, que forman la subllave K_i.

Descifrado

Gracias a la estructura de Feistel, descifrar es el mismo algoritmo con las subllaves en orden inverso, de K_{16} a K_1. La función f no tiene que ser invertible, porque cada ronda se deshace aplicando otra vez el mismo XOR (Stallings, 2020).

Modo ECB y relleno

DES cifra bloques de 8 bytes. Para textos más largos usamos el modo ECB (*Electronic Codebook*), que cifra cada bloque por separado con la misma llave. Nuestro texto, «Esta es una prueba de proyecto 2», mide 32 bytes, es decir, 4 bloques exactos. Si no fuera múltiplo de 8, el último bloque se rellenaría con ceros (función `padded_length`).

ECB no es seguro para mensajes reales. Dos bloques de texto iguales producen dos bloques cifrados iguales, y así se filtran patrones del mensaje. Los modos encadenados como CBC o CTR lo evitan (Stallings, 2020). En este proyecto no importa, porque lo que se ataca es la llave.

DES ante la fuerza bruta

Con 56 bits efectivos, DES quedó al alcance del *hardware* dedicado a finales de los noventa. En 1998, la máquina Deep Crack de la Electronic Frontier Foundation, que costó unos 250 000 dólares, recuperó una llave en 56 horas. En enero de 1999, el DES Challenge III, resuelto por la EFF junto con distributed.net, lo logró en 22 horas y 15 minutos (EFF, 1998).

Nuestro programa secuencial prueba unos $3 \cdot 10^6$ llaves por segundo en la laptop de Esteban, la máquina de las mediciones (sección A.3). A ese ritmo, recorrer las 2^{56} llaves le tomaría unos 770 años, y en promedio la mitad. Por eso el proyecto paraleliza la búsqueda y, para medir, reduce el espacio con la opción `-b`.

A.2 Diagrama de flujo

La figura 1 resume el cifrado de un bloque y la generación de las 16 subllaves. El diagrama se generó con Mermaid desde `docs/img/des-flujo.mmd`.

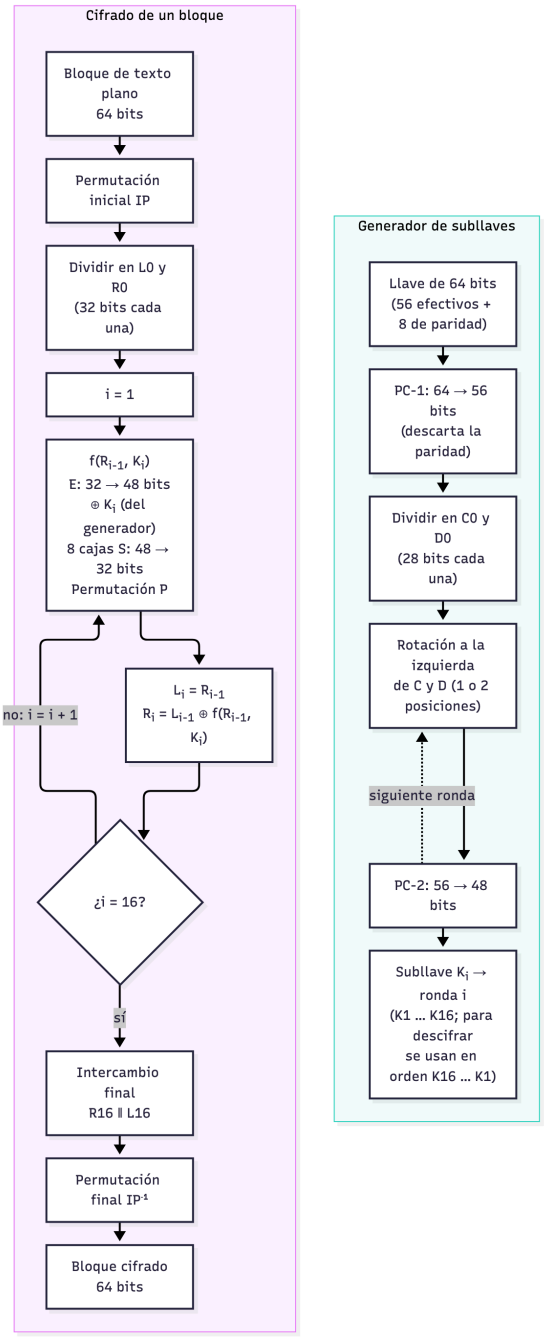


Figura 1. Cifrado de un bloque con DES (izquierda) y generación de las 16 subllaves (derecha).

A.3 Búsqueda secuencial con distintas longitudes de llave

El programa `secuencial` prueba las llaves 0, 1, 2 y así en orden, hasta que el texto descifrado contiene la frase clave. Para medir cómo crece el tiempo con la longitud de la llave se usó el peor caso de cada longitud: con b bits, la llave $2^b - 1$, que obliga a probar las 2^b llaves.

Las corridas las hace el script `medir_longitudes.sh`. En la laptop de Esteban cada longitud se repitió 5 veces y se reporta la mediana. En la Mac de Nico hubo una sola corrida por longitud, que sirve como comparación. Los datos crudos están en la carpeta `resultados/`, en los archivos `secuencial_longitudes_escu.csv` (laptop) y `secuencial_longitudes.csv` (Mac).

Bits	Llaves probadas	Laptop de Esteban (s)	Ritmo (llaves/s)	Mac de Nico (s)
16	65 536	0.044	$1.50 \cdot 10^6$	0.020
18	262 144	0.112	$2.35 \cdot 10^6$	0.071
20	1 048 576	0.370	$2.83 \cdot 10^6$	0.272
22	4 194 304	1.370	$3.06 \cdot 10^6$	1.082
24	16 777 216	5.437	$3.09 \cdot 10^6$	4.322
26	67 108 864	21.939	$3.06 \cdot 10^6$	17.241

El tiempo crece de forma exponencial con la longitud de la llave. Cada bit duplica el espacio, así que cada 2 bits el tiempo se multiplica por 4: de 22 a 24 bits pasa de 1.37 s a 5.44 s, y de 24 a 26 bits, a 21.94 s. Dicho de otro modo, el tiempo es lineal en el número de llaves, porque cada intento cuesta lo mismo.

Con 16 y 18 bits el ritmo es menor. Esas corridas duran milisegundos, y en ese tiempo la CPU todavía no alcanza su frecuencia máxima. Desde 22 bits el ritmo se estabiliza en unos $3.1 \cdot 10^6$ llaves por segundo, y ese es el valor que sirve para extrapolar.

Con ese ritmo, 28 bits tomarían unos 87 s en el peor caso, y la llave media de la campaña, que está en el 62.5 % del espacio, unos 54 s; la campaña midió 56.6 s. Con 56 bits, el peor caso tomaría $2^{56} / 3.1 \cdot 10^6 \approx 2.3 \cdot 10^{10}$ s, más de 700 años. La Mac es alrededor de un 27 % más rápida, pero eso no cambia el orden de magnitud. Por eso las mediciones del proyecto se hacen con 28 bits.

Parte A.4 a A.6: Rutinas de la base y primitivas MPI

A.4 Por qué no compila `rpc/des_crypt.h`

El `bruteforce.c` del curso incluye `<rpc/des_crypt.h>` y usa `ecb_crypt` y `des_setparity`. Esas funciones eran parte de la implementación de Sun RPC que traía la biblioteca de C de GNU (glibc). En glibc 2.32, publicada en 2020, se eliminó por completo Sun RPC junto con `rpcgen` y sus cabeceras (Fedora Project, s. f.; Weimer, 2020). Desde entonces se recomienda la biblioteca externa libtirpc, que no viene instalada por defecto.

El motivo fue que esa implementación no soportaba IPv6, y arreglarlo exigía romper la ABI (Fedora Project, s. f.). Por eso, en una distribución moderna como Ubuntu 22.04, Arch o Fedora, la línea `#include <rpc/des_crypt.h>` falla con «No such file or directory». En macOS tampoco existe.

En lugar de depender de libtirpc, usamos la API DES de OpenSSL (`<openssl/des.h>`, enlazada con `-lcrypto`). Está en cualquier sistema y permite verificar el resultado con `openssl enc -des-ecb` (OpenSSL Project, s. f.). OpenSSL 3 marca esa API como obsoleta porque DES ya no es seguro, pero la mantiene; el aviso se apaga con `-DOPENSSL_SUPPRESS_DEPRECATED`. La tabla siguiente muestra qué rutina de nuestro módulo `des_utils` reemplaza a cada original.

`bruteforce.c` (Sun RPC)	Nuestro proyecto (OpenSSL)
`encrypt(key, ciph, len)`	`encrypt_des(key, buffer, length)`
`decrypt(key, ciph, len)`	`decrypt_des(key, buffer, length)`
`tryKey(key, ciph, len)`	`try_key(key, cipher, length, phrase, workspace)`
`des_setparity(k)`	`DES_set_odd_parity(&k)`
`ecb_crypt(k, buf, len, DES_ENCRYPT)`	`DES_set_key_unchecked` + `DES_ecb_encrypt` por bloque

A.5 Las rutinas de la base

`encrypt_des` y `decrypt_des`

Ambas llaman a la misma función interna, `run_des`, y solo cambian la dirección (`DES_ENCRYPT` o `DES_DECRYPT`). Trabajan sobre el mismo buffer: el resultado sobrescribe la entrada.

Lo primero es armar la llave con `build_des_key`. La llave es un número de 56 bits, pero DES pide 8 bytes en los que el bit menos significativo de cada uno es de paridad. Igual que el `decrypt()` original, los 56 bits se reparten en grupos de 7: el byte i guarda los bits 7i a 7i + 6 en sus posiciones 1 a 7. Después, `DES_set_odd_parity` llena la posición 0 para que cada byte tenga paridad impar.

Con la llave lista, `DES_set_key_unchecked` calcula las 16 subllaves de 48 bits con PC-1, las rotaciones y PC-2. Usamos la versión sin revisión porque la normal rechaza las llaves débiles de DES, y en fuerza bruta hay que probarlas todas. Por último, `DES_ecb_encrypt` procesa cada bloque de 8 bytes con la misma agenda. La longitud siempre es múltiplo de 8 gracias al relleno con ceros.

`try_key`

Es el corazón de la búsqueda, porque prueba una llave candidata. Copia el cifrado al buffer `workspace` con `memcpy` y le pone un `'\0'` al final para tratarlo como cadena de C. Luego lo descifra con la llave candidata y busca la frase clave con `strstr`.

Si la frase aparece, la llave es la correcta. Con muy poca probabilidad podría ser un falso positivo, y en ese caso el reporte lo marca. El `workspace` lo reserva una sola vez quien llama, en lugar de pedir memoria en cada intento como hacía el original. Así se evitan millones de llamadas a `malloc` y `free`.

`memcpy`

`memcpy(destino, origen, n)` copia `n` bytes de un buffer a otro (ISO/IEC, 2011). Aquí es indispensable porque `decrypt_des` descifra sobre el mismo buffer. Si se descifrara el cifrado original, el primer intento fallido lo destruiría y no quedaría nada que probar con la llave siguiente. Con la copia, el cifrado original queda intacto durante toda la búsqueda.

`strstr`

`strstr(texto, frase)` devuelve un puntero a la primera aparición de `frase` dentro de `texto`, o `NULL` si no aparece (ISO/IEC, 2011). Es la prueba de que la llave es correcta: con una llave errónea, el descifrado es basura y la frase no aparece. Por eso el texto debe terminar en `'\0'`, y por eso `problem_prepare` rechaza de entrada una frase que no esté en el texto original. La figura 2 muestra la secuencia completa de un intento.

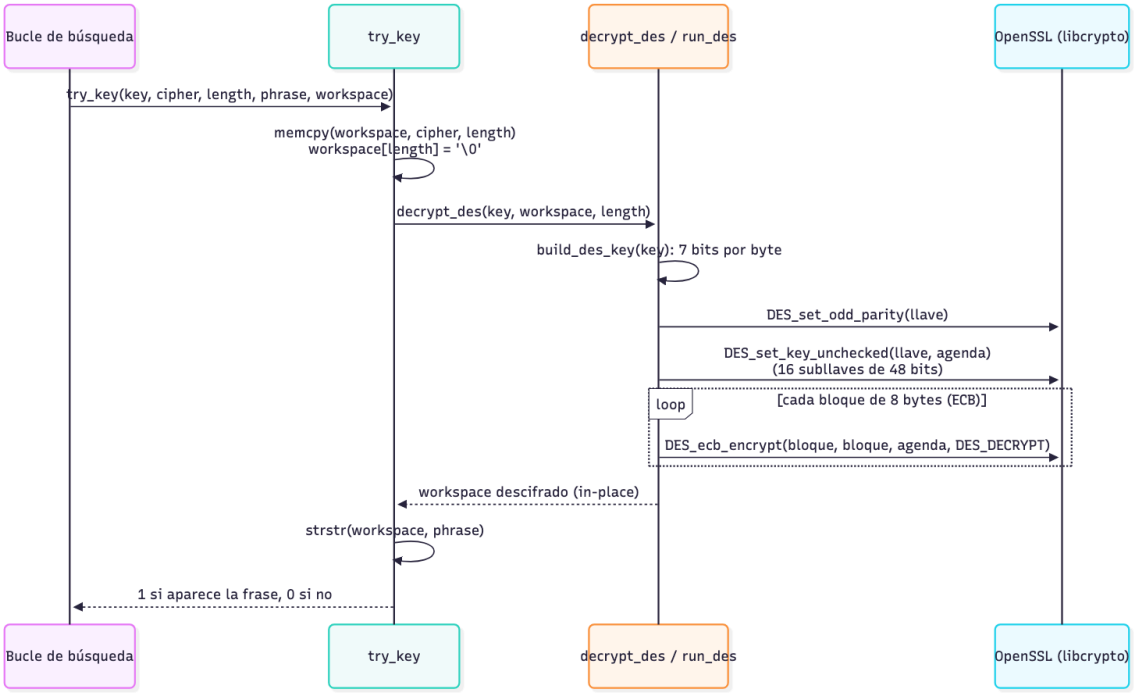


Figura 2. Lo que ocurre en cada llamada a `try_key`.

A.6 Primitivas MPI del aviso de llave

El aviso de que alguien halló la llave usa cuatro primitivas de MPI (MPI Forum, 2023). La tabla resume para qué sirve cada una.

Primitiva	Tipo	Para qué la usamos
`MPI_Irecv`	No bloqueante	Dejar puesta una «antena» que reciba el aviso de llave encontrada
`MPI_Send`	Bloqueante	Que quien halla la llave avise a todos los procesos
`MPI_Wait`	Bloqueante	Cerrar la recepción pendiente antes de terminar
`MPI_Test`	No bloqueante	Revisar, sin detenerse, si ya llegó el aviso

`MPI_Irecv`

`MPI_Irecv(buf, count, datatype, source, tag, comm, &request)` inicia una recepción y regresa de inmediato, sin esperar el mensaje. Devuelve un `MPI_Request` con el que después se consulta o se espera. En el naive, cada proceso la publica antes de empezar a buscar:

```
MPI_Irecv(found_msg, 3, MPI_UINT64_T, MPI_ANY_SOURCE, FOUND_TAG, MPI_COMM_WORLD,
&request);
```

Con `MPI_ANY_SOURCE` el aviso se recibe venga de quien venga, y `found_msg` guarda la llave, el proceso que la halló y sus iteraciones. Mientras tanto, el proceso sigue probando llaves; por eso la llamamos la antena.

`MPI_Send`

`MPI_Send(buf, count, datatype, dest, tag, comm)` envía un mensaje y no regresa hasta que el buffer se puede reutilizar. El proceso que encuentra la llave envía el aviso a todos, incluido él mismo, para que su propio `MPI_Irecv` también se complete y todos cierren igual. El mensaje mide 24 bytes y cada destino ya tiene su recepción publicada, así que el envío termina casi de inmediato.

`MPI_Wait`

`MPI_Wait(&request, &status)` bloquea hasta que termina la operación no bloqueante asociada. Al salir del bucle, todos los procesos hacen `MPI_Wait` sobre su antena. Así se aseguran de que `found_msg` ya tiene el aviso antes de usarlo y de que no queda nada pendiente al llamar a `MPI_Finalize`. Si un proceso agotó su bloque sin hallar nada, `MPI_Wait` lo deja esperando hasta que otro avise.

Por qué hace falta `MPI_Test`

El `bruteforce.c` original revisa una variable `found` en cada vuelta, pero esa variable solo cambia si la biblioteca MPI hace avanzar la recepción. Open MPI no tiene, por defecto, un hilo que procese mensajes en segundo plano: las comunicaciones solo avanzan dentro de llamadas a MPI. Un proceso que solo ejecuta `try_key` nunca entra a MPI, así que el aviso no se entrega y el proceso termina su bloque completo.

Por eso, cada `POLL_INTERVAL` = 4096 llaves, el naive llama a `MPI_Test(&request, &done, MPI_STATUS_IGNORE)`. Esa llamada le da a MPI la oportunidad de avanzar y, si el aviso ya llegó, `done` vale 1 y el proceso sale del bucle. Probar 4096 llaves toma alrededor de 1 ms, así que el costo es despreciable y el retraso para enterarse es de ese orden.

Flujo de mensajes del naive con 4 procesos

En la figura 3, el proceso 1 encuentra la llave y envía el aviso a los 4 procesos. Cada uno lo detecta en su siguiente `MPI_Test` y cierra con `MPI_Wait`. Al final, el proceso 0 reúne las iteraciones con `MPI_Reduce` y reporta el resultado.

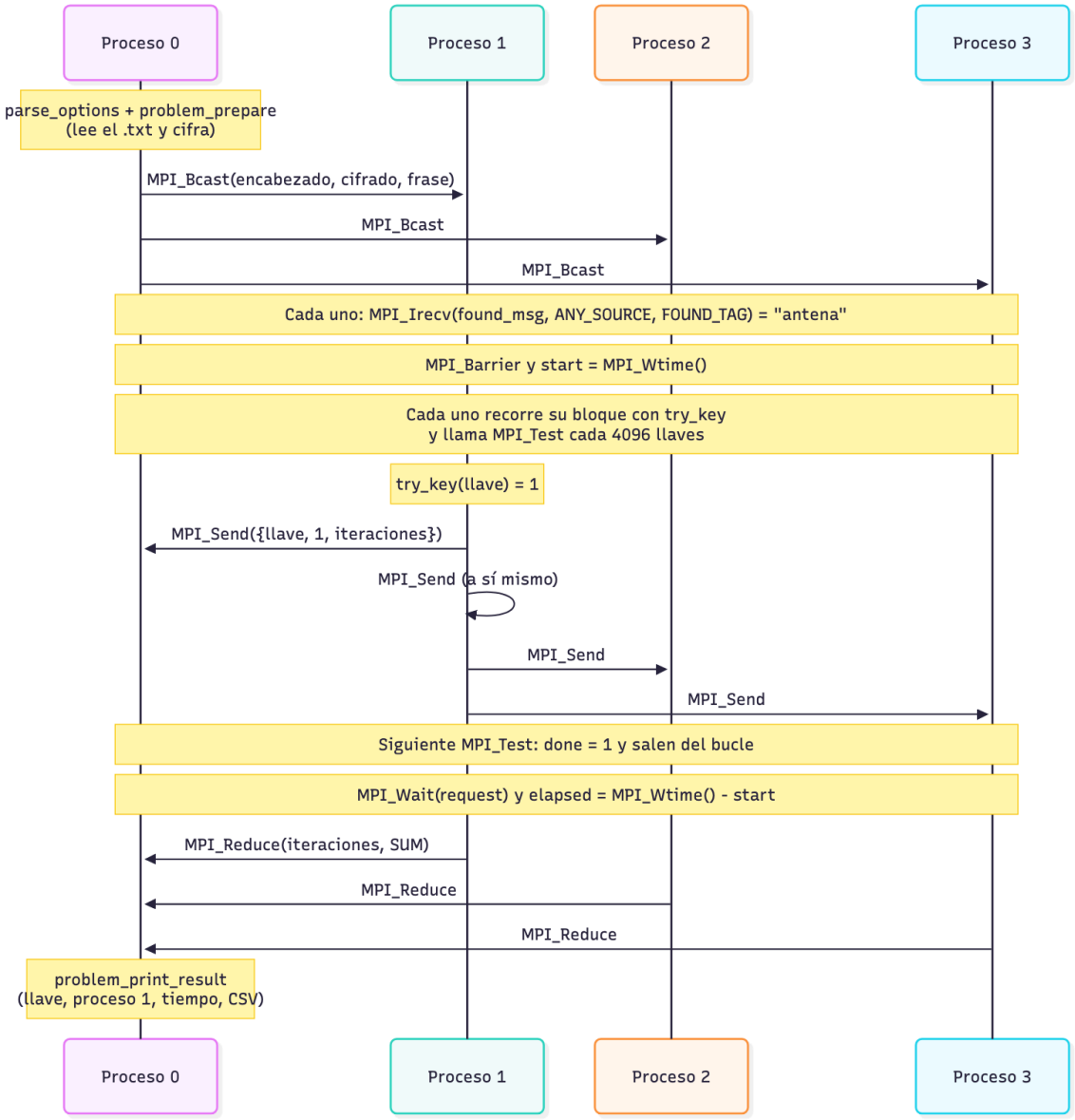


Figura 3. Mensajes del naive con 4 procesos cuando el proceso 1 halla la llave.

Además del aviso, el naive usa `MPI_Bcast` para repartir el problema, porque solo el proceso 0 lee el archivo y cifra. Usa `MPI_Barrier` para que todos arranquen el reloj (`MPI_Wtime`) a la vez y `MPI_Reduce` para sumar las llaves probadas. Cuando hay límite de tiempo (`-t`), el proceso 0 envía el mismo aviso con la llave imposible `UINT64_MAX`, que el reporte traduce a «no encontrada».

Parte B.1 y B.2: Pruebas del cifrado y del naive

Todas las corridas de este capítulo se hicieron en la MacBook Pro de Nico: Apple M4 Pro con 8 núcleos de rendimiento y 4 de eficiencia, macOS 26.6.1, Open MPI 5.0.11 y OpenSSL 3 de Homebrew. Se usó el texto `data/texto.txt` con la frase clave «es una prueba de». Los datos crudos están en `resultados/pruebas_b2.csv` y los genera el script `scripts/pruebas_b2.sh`.

B.1 Cifrar y descifrar desde un `.txt`

El programa `cifrar` carga el texto de `data/texto.txt`, lo cifra con la llave 42 que recibe por parámetro (`-k 42`) y lo descifra de vuelta. El texto descifrado coincide con el original, como se ve en la figura 4.

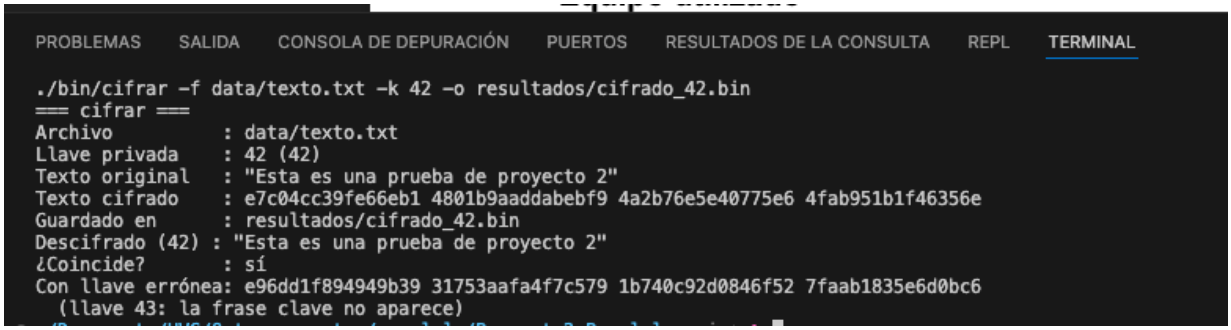


Figura 4. Cifrado y descifrado de `data/texto.txt` con la llave 42.

El cifrado, `e7c04cc39fe66eb1 4801b9aaddabebf9 4a2b76e5e40775e6 4fab951b1f46356e`, es el mismo que produce `openssl enc -des-ecb` con esa llave. Con la llave vecina, la 43, sale basura y la frase clave no aparece. Eso es justo lo que la búsqueda usa para distinguir la llave correcta.

B.2 Naive con 4 procesos contra el secuencial

Las tres llaves del enunciado, con $N = 2^{56}$, caen en lugares muy distintos del reparto. Con `-np 4`, el naive da a cada proceso un bloque contiguo de $N/4 = 2^{54}$ llaves: el proceso 0 prueba de 0 a $N/4 - 1$, el proceso 1 de $N/4$ a $N/2 - 1$, y así sucesivamente.

Prueba	Llave	Posición
a	123456	Muy al inicio del espacio, dentro del bloque del proceso 0
b	$N/4 - 1$ = 18014398509481983	Última llave del bloque del proceso 0
c	$N/4$ = 18014398509481984	Primera llave del bloque del proceso 1

Resultados con 56 bits

Con 56 bits se fijó un límite de 120 s por corrida. La tabla muestra qué programa halló la llave y en cuánto tiempo.

Prueba	Programa	¿La halló?	Proceso	Iteraciones	Llaves probadas	Tiempo
a	secuencial	sí	0	123 457	123 457	0.054 s
a	naive -np 4	sí	0	123 457	492 097	0.034 s
b	secuencial	no (límite)	—	—	459 493 376	120.0 s
b	naive -np 4	no (límite)	—	—	1 689 812 992	120.0 s
c	secuencial	no (límite)	—	—	433 438 720	120.0 s
c	naive -np 4	sí	1	1	12 289	0.0012 s

La prueba a es tan corta que la primera corrida tiene ruido de arranque. Repetida 5 veces, la mediana es 0.032 s para el secuencial y 0.034 s para el naive.

Con 56 bits, las pruebas b y c no terminan en el secuencial, así que su tiempo se estima. En la corrida de 120 s el secuencial probó $r_s \approx 3.83 \cdot 10^6$ llaves/s. Para b, $T_s \approx 2^{54} / r_s \approx 4.70 \cdot 10^9$ s, unos 149 años. Para c el cálculo es prácticamente el mismo.

En el naive, la llave b es la última del bloque del proceso 0, que la alcanza tras 2^{54} intentos. Cada proceso probó unas $3.52 \cdot 10^6$ llaves/s (1 689 812 992 / 4 / 120 s). Entonces $T_p(b) \approx 2^{54} / 3.52 \cdot 10^6 \approx 5.1 \cdot 10^9$ s, unos 162 años.

Resultados con 28 bits

Con `-b 28` los presets escalan (b = $2^{26} - 1$ y c = 2^{26}) y conservan su posición relativa. La llave b sigue siendo la última del proceso 0 y la c, la primera del proceso 1.

Prueba	Programa	Proceso	Iteraciones	Tiempo
b	secuencial	0	67 108 864	18.20 s
b	naive -np 4	0	67 108 864	19.00 s
c	secuencial	0	67 108 865	18.56 s
c	naive -np 4	1	1	0.0012 s

Speedup

El speedup es $S = T_s / T_p$. Además reportamos el speedup de rendimiento, S_{rend}, que divide las llaves por segundo del naive entre las del secuencial. Ese segundo valor mide cuánto trabajo útil hacen los 4 procesos juntos, sin importar dónde cae la llave.

Prueba	T_s	T_p (naive -np 4)	$S = T_s / T_p$	Comentario
a · 56 bits	0.032 s	0.034 s	≈ 0.9	El proceso 0 hace el mismo trabajo que el secuencial
b · 56 bits	≈ 149 años (est.)	≈ 162 años (est.)	≈ 0.9	Ninguna ganancia
c · 56 bits	≈ 149 años (est.)	0.0012 s	≈ $3.9 \cdot 10^{12}$	Aparente: efecto de búsqueda
b · 28 bits	18.20 s	19.00 s	0.96	Ninguna ganancia, medido completo
c · 28 bits	18.56 s	0.0012 s	≈ $1.6 \cdot 10^{4}$	Aparente: efecto de búsqueda

En la prueba b con 56 bits, los 4 procesos probaron juntos $14.1 \cdot 10^6$ llaves/s, contra $3.83 \cdot 10^6$ del secuencial. Eso da $S_{rend} \approx 3.7$, que no pasa de p = 4. El naive sí trabaja casi cuatro veces más rápido; lo que pasa es que en b todo ese trabajo extra se hace en bloques donde la llave no está.

Reflexión

El tiempo del naive no depende de cuántas llaves hay que probar en total. Depende de dónde cae la llave dentro del bloque del proceso que la tiene: $T_p \approx$ (llave − inicio de su bloque + 1) dividido entre el ritmo de un proceso.

La llave 123456 de la prueba a cae al inicio del bloque del proceso 0. Ese proceso hace los mismos 123 457 intentos que el secuencial, y los otros tres no aportan nada. El speedup ronda 1: paralelizar no ayudó y solo agregó el costo de arrancar MPI.

La llave de la prueba b es la última del proceso 0. Ese proceso recorre sus 2^{54} llaves en unos 162 años, igual que el secuencial en 149, mientras los otros tres terminan sus bloques sin encontrar nada. El speedup vuelve a rondar 1, o un poco menos por el costo de `MPI_Test`.

La llave de la prueba c es la primera del proceso 1, que la encuentra en un solo intento y en un milisegundo. El secuencial tardaría 149 años, así que el speedup es de unos $4 \cdot 10^{12}$, muchísimo mayor que p = 4.

Las llaves b y c son vecinas, pues difieren en 1. Sin embargo, una da el peor resultado posible y la otra un speedup astronómico. Ese S > p de la llave c no es superlinealidad real: es el efecto de búsqueda. El naive no hizo el trabajo más rápido; simplemente empezó a buscar justo donde estaba la llave. El speedup de rendimiento, que en nuestras corridas no pasa de 3.7, muestra lo que el paralelismo aporta de verdad.

Esto deja ver la debilidad del reparto en bloques contiguos: el resultado depende de la suerte y no del algoritmo. Da una «falsa confianza» si se prueba con llaves al inicio de un bloque, y no ayuda nada si la llave cae al final del primero. Los enfoques alternativos del capítulo siguiente reparten las llaves para que todos los procesos avancen a la vez por las zonas bajas del espacio. Así el speedup se vuelve consistente y cercano a p para cualquier llave.

Capturas

Para las capturas, las corridas que no terminan usan un límite de 20 s en lugar de 120 s. Las tablas anteriores salen de la corrida completa de `scripts/pruebas_b2.sh`.

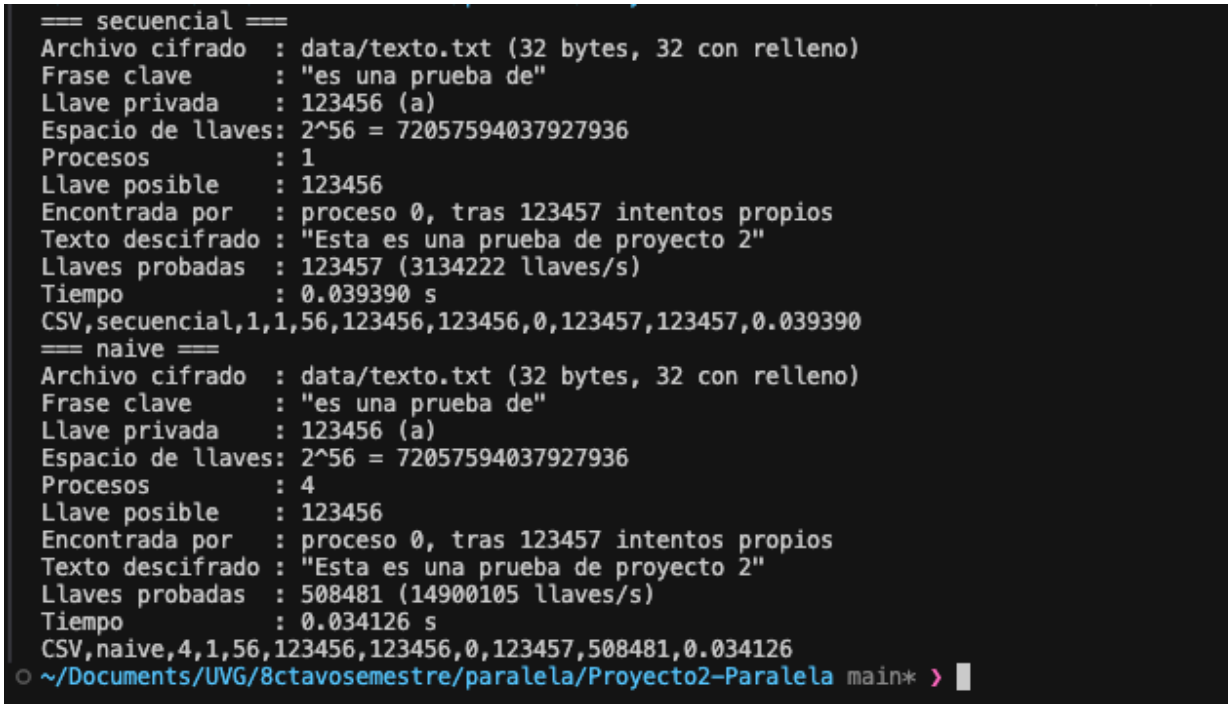


Figura 5. Prueba a: el secuencial y el naive hallan la llave 123456 en el proceso 0 tras 123 457 intentos.

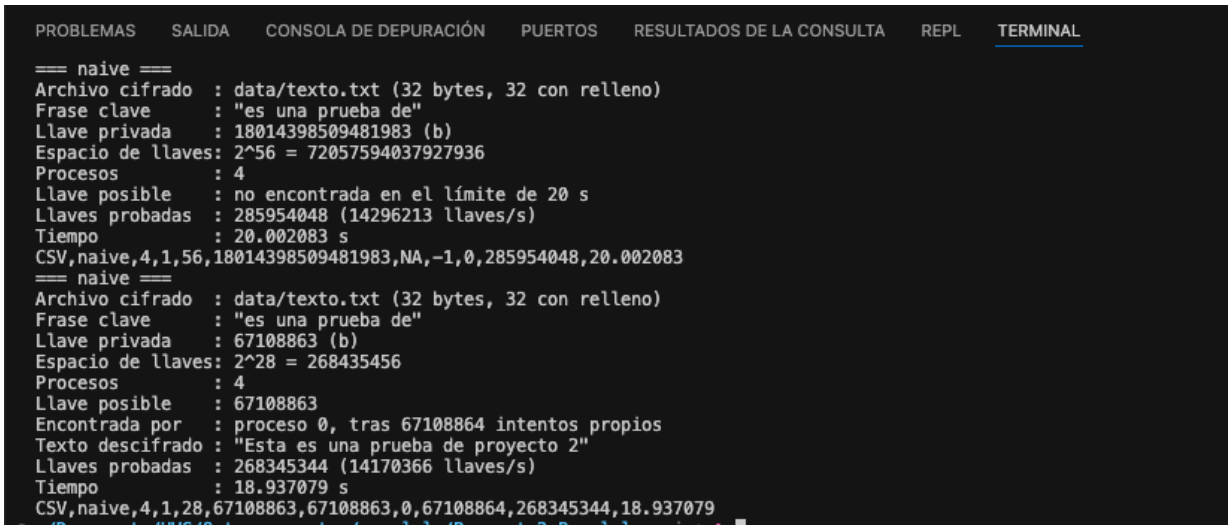


Figura 6. Prueba b: con 56 bits el naive no la halla en el límite; con 28 bits la halla el proceso 0 tras recorrer su bloque completo (2^{26} intentos, 18.9 s).

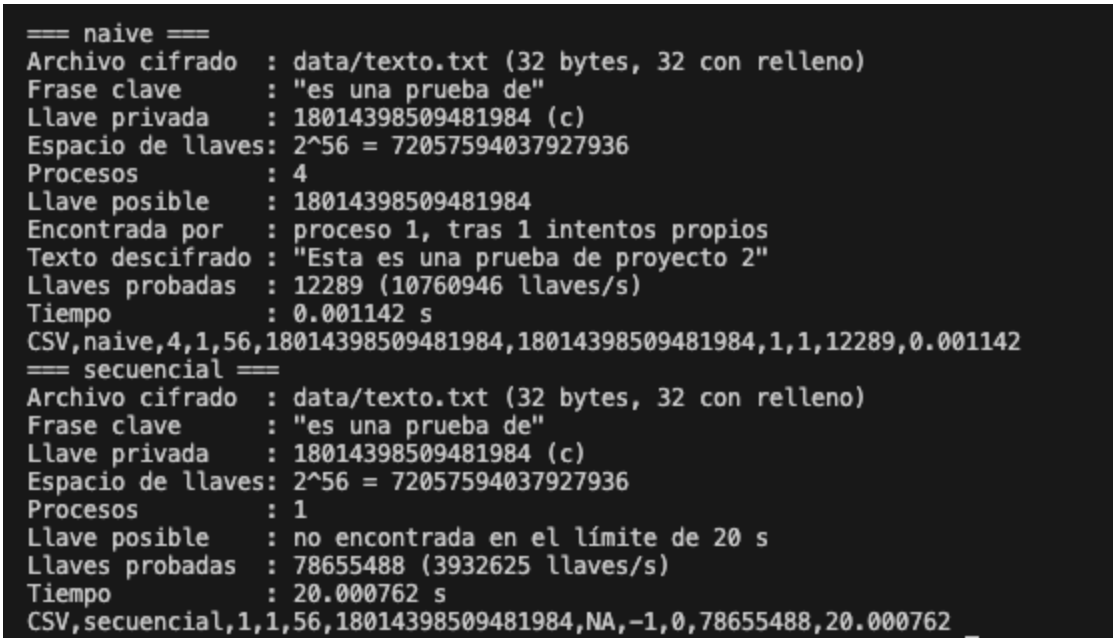


Figura 7. Prueba c: el proceso 1 halla la llave al primer intento (0.001 s), mientras el secuencial no la halla en el límite.

Enfoques de paralelización

Los programas MPI resuelven el mismo problema con el mismo arranque, el mismo aviso de llave y el mismo reporte, que comparten en el módulo `search_mpi`. Lo único que cambia entre ellos es qué llaves prueba cada proceso y en qué orden. Este capítulo describe cada reparto y predice con una fórmula quién halla la llave y en cuántas iteraciones. También discute qué gana y qué pierde cada uno.

Esqueleto común

Todos los repartos estáticos siguen el mismo esqueleto. Solo el proceso 0 lee y cifra el texto, y luego lo difunde a los demás. Cada proceso deja puesta su antena de aviso antes de buscar y arranca el reloj después de una barrera.

```
proceso 0: lee parámetros y archivo, cifra el texto
MPI_Bcast(encabezado, cifrado, frase)           # errores: salen una vez y todos
cierran
MPI_Irecv(aviso)                                # la "antena", puesta antes de
buscar
MPI_Barrier; inicio = MPI_Wtime()
para cada llave k del reparto de este proceso:
    cada POLL_INTERVAL = 4096 llaves: MPI_Test(antena); si llegó el aviso, salir
    si try_key(k): MPI_Send(aviso {k, rango, iteraciones}) a todos; salir
MPI_Wait(antena); T_p = MPI_Wtime() - inicio
MPI_Reduce(llaves probadas, SUMA) y el proceso 0 reporta
```

El sondeo cada 4096 llaves es necesario porque Open MPI no hace avanzar la recepción mientras el proceso calcula. Su costo es despreciable, una llamada por cada 4096 descifrados. El aviso se envía también a uno mismo, para que su propio `MPI_Irecv` se complete y el `MPI_Wait` final no quede esperando.

En lo que sigue, $N = 2^{bits}$ es el número de llaves, p el de procesos y k la llave buscada. Las iteraciones son las llaves que probó el proceso que halló la llave. El secuencial prueba k + 1 llaves, así que cada reparto predice, sin necesidad de medir, un speedup por iteraciones S_{iter} = (k + 1) / (iteraciones del que la halla).

La figura 8 compara qué llaves prueba cada proceso en los tres repartos estáticos, con un espacio pequeño de 16 llaves.

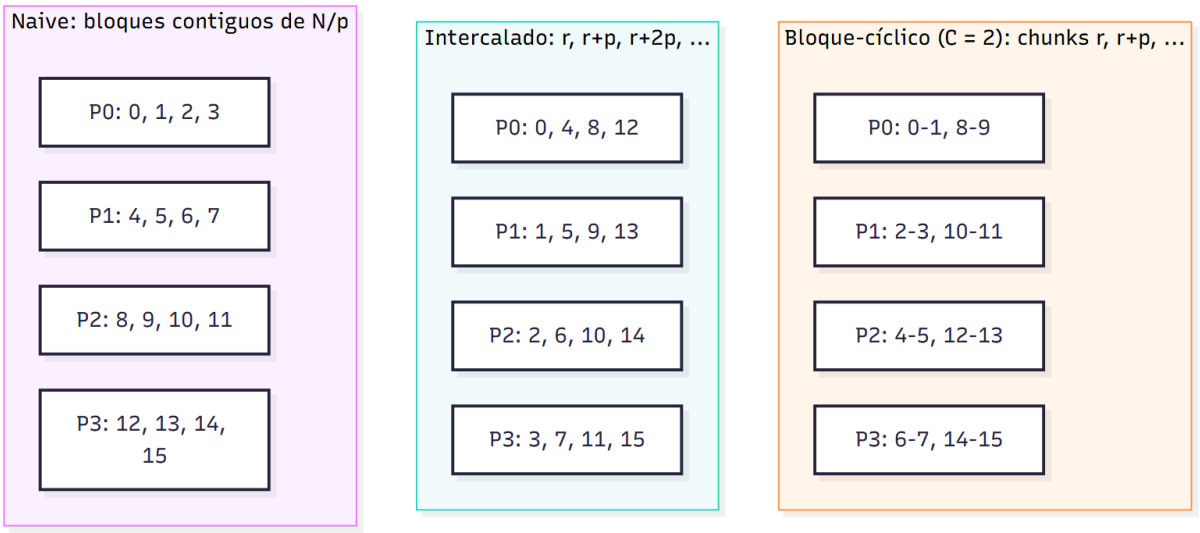


Figura 8. Llaves que prueba cada proceso en los repartos estáticos, con 16 llaves, p = 4 y chunks de C = 2.

Naive: bloques contiguos

El naive es el reparto del `bruteforce.c` del curso y sirve de referencia. El espacio se parte en p bloques de N/p llaves, y el proceso r recorre el suyo de principio a fin.

```
bloque = N / p
inicio = r · bloque;  fin = (r == p - 1) ? N : inicio + bloque
para k = inicio .. fin - 1: probar k
```

La llave la halla el proceso $r = \lfloor k / (N/p) \rfloor$, tras $k - r \cdot N/p + 1$ iteraciones. Por eso $S_{iter} = (k + 1) / (k - r \cdot N/p + 1)$. Ese valor va de 1, cuando la llave cae en el bloque del proceso 0, a cerca de N/p, cuando la llave es la primera de un bloque que no es el primero.

Su ventaja es la simpleza: recorre llaves contiguas y no necesita más comunicación que el aviso. Su desventaja es que el speedup depende por completo de dónde cae la llave. Con llaves pequeñas, que son las que más probablemente elige una persona, no gana nada.

La fuga de eficiencia que domina es el desbalance respecto a la llave. Los $p - 1$ procesos que no la tienen trabajan en vano, y el que la tiene recorre, en promedio, la mitad de su bloque.

Intercalado

En el intercalado, el proceso r prueba las llaves r, r + p, r + 2p, y así hasta el final. Todos los procesos avanzan a la vez por las llaves bajas y por las altas.

```
para k = r, r + p, r + 2p, ... < N: probar k
```

La llave la halla el proceso k mod p, tras $\lfloor k/p \rfloor + 1$ iteraciones. Entonces $S_{iter} = (k + 1) / (\lfloor k/p \rfloor + 1)$, que vale casi exactamente p y nunca lo supera.

Su gran ventaja es que el speedup ronda p para cualquier llave, sin comunicación extra y cambiando solo dos líneas respecto al naive. Como desventaja, cada proceso salta de p en p y no aprovecha llaves contiguas, aunque aquí eso no importa porque cada llave genera su propia agenda de subllaves. Además es un reparto estático: si un núcleo es más lento, todos lo esperan.

Su comunicación es mínima, solo el sondeo y el aviso. La pérdida que domina la pone el hardware: con más núcleos activos baja el turbo, y tanto el i9 como el Apple Silicon mezclan núcleos de rendimiento y de eficiencia.

Bloque-cíclico

El bloque-cíclico parte el espacio en chunks de $C = 2^c$ llaves, con `-c` igual a 16 por defecto. El proceso r recorre los chunks r, r + p, r + 2p, y dentro de cada uno avanza en orden. Es el punto medio entre los dos anteriores: con C = N/p es el naive y con C = 1 es el intercalado.

```
para j = r, r + p, r + 2p, ... < N / C:
    para k = j·C .. (j + 1)·C - 1: probar k
```

Si $j = \lfloor k/C \rfloor$ es el chunk de la llave, la halla el proceso j mod p tras $\lfloor j/p \rfloor \cdot C + (k \bmod C) + 1$ iteraciones. El speedup se acerca a p cuando k es mucho mayor que p·C. Con llaves diminutas se parece al naive: la llave 123456, con $C = 2^{16}$, cae en el chunk 1, del proceso 1, en la posición 57 920, y da $S_{iter} \approx 2.13$.

Este reparto combina el equilibrio del intercalado con el recorrido contiguo del naive, y el tamaño de chunk permite moverse entre ambos. A cambio, agrega un parámetro, y con chunks grandes respecto a la llave se pierde la consistencia. Su fuga principal es el desbalance dentro del último chunk, de como mucho C llaves, más la misma comunicación mínima del intercalado.

Maestro-trabajador

En este enfoque, el proceso 0 es el maestro y no busca: reparte chunks de C llaves a quien los pide. Los procesos 1 a p − 1 son trabajadores, que piden un chunk, lo prueban y piden otro. Es el único reparto dinámico, porque un núcleo más lento simplemente pide menos chunks.

```
maestro:                                    trabajador:
  activos = p - 1                             pedir; recibir respuesta
  mientras activos > 0:                       mientras la respuesta sea WORK:
    recibir pedido (MPI_ANY_SOURCE)             pedir el siguiente (MPI_Send +
MPI_Irecv)
    si es mensaje final (FOUND o DONE):         probar el chunk actual
      si FOUND: T_p = ahora; parar = 1            si halló k: enviar FOUND {k, iter}
      activos -= 1                                          MPI_Wait(respuesta) y
terminar
    si no: responder STOP si parar o no       MPI_Wait(respuesta adelantada)
           quedan chunks; si no, WORK         enviar DONE
```

Dos invariantes evitan cuelgues. Cada pedido recibe exactamente una respuesta, y cada trabajador termina con un único mensaje final, FOUND o DONE, que no espera respuesta. Como MPI no adelanta mensajes del mismo origen con la misma etiqueta, cuando el maestro lee el mensaje final de un trabajador ya atendió todos sus pedidos anteriores.

El trabajador pide el chunk siguiente antes de probar el actual. Ese doble buffer, con `MPI_Send`, `MPI_Irecv` y `MPI_Wait`, esconde la ida y vuelta con el maestro detrás del cálculo. La figura 9 muestra el intercambio de mensajes con tres procesos.

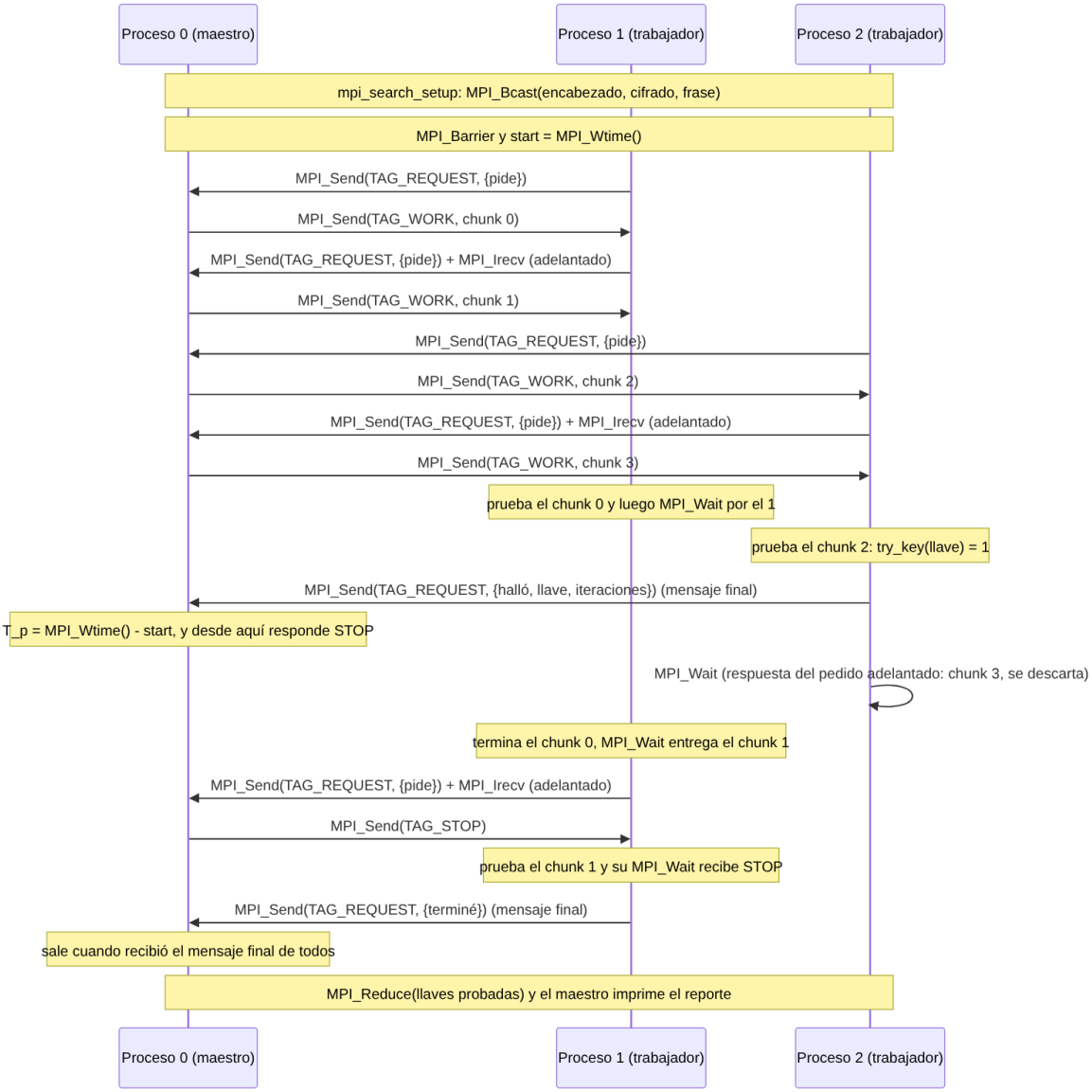


Figura 9. Mensajes del maestro-trabajador con un maestro y dos trabajadores.

Con $p - 1$ trabajadores que van al mismo ritmo, los chunks se reparten casi en round-robin entre ellos. Por eso se comporta como el bloque-cíclico con $p - 1$ procesos: $S_{iter} \approx p - 1$, unos 3 con `-np 4`, y su eficiencia ideal es $(p - 1)/p$, 0.75 con `-np 4`.

Su ventaja es el balanceo automático cuando los núcleos son desiguales o la máquina está compartida. Además respeta el límite `-t` con una resolución de un chunk. Su desventaja es que el maestro no busca, así que con pocos procesos se pierde 1/p del poder de cómputo. Con `-np 2` hay un solo trabajador y no puede ganarle al secuencial.

Su fuga principal es una parte secuencial estructural, el maestro que solo reparte. Le sigue la comunicación, un pedido y una respuesta por chunk, oculta en parte por el pedido

adelantado. Con más procesos, (p − 1)/p se acerca a 1.

Híbrido MPI + OpenMP

El híbrido es el intercalado con dos niveles: p procesos MPI y T hilos de OpenMP por proceso, según `OMP_NUM_THREADS`. El hilo t del proceso r es el trabajador global g = r·T + t, y prueba g, g + pT, g + 2pT, y así sucesivamente. Es el mismo intercalado, ahora con w = p·T trabajadores, como muestra la figura 10.

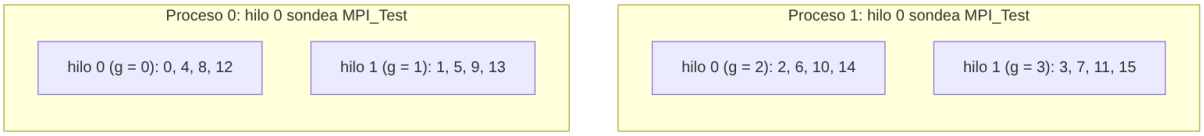


Figura 10. Llaves que prueba cada hilo del híbrido con 16 llaves, p = 2 y T = 2.

```
MPI_Init_thread(MPI_THREAD_FUNNELED)            # solo el hilo principal llama a
MPI
mpi_search_setup; T = omp_get_max_threads() del proceso 0, MPI_Bcast(T)
antena: MPI_Irecv; MPI_Barrier; inicio = MPI_Wtime()
#pragma omp parallel num_threads(T) reduction(+: llaves)
    g = r·T + t;  workspace propio
    para k = g, g + pT, g + 2pT, ... < N:
        cada 4096 llaves:
            si t == 0 y MPI_Test(antena): stop = 1     # aviso de otro proceso
            si stop: salir
        si try_key(k): critical { guardar k, iteraciones }; stop = 1; salir
si un hilo la halló: MPI_Send(aviso) a todos        # fuera de la región paralela
MPI_Wait(antena); MPI_Reduce(llaves); reporte con hilos = T
```

La función `try_key` es reentrante, porque la agenda de subllaves de DES es local, así que basta un `workspace` por hilo. La bandera de parada es un `atomic_int` de C11, y los hilos la leen cada 4096 llaves, igual que el sondeo MPI.

MPI se inicia con `MPI_THREAD_FUNNELED`. Solo el hilo 0, que es el principal, llama a `MPI_Test`, y el aviso sale después de la región paralela. Así no hace falta que el MPI instalado acepte llamadas desde varios hilos. El número de hilos lo decide el proceso 0 y lo difunde, para que todos usen el mismo paso p·T.

Al medir hay un detalle importante. Open MPI 5 ata cada proceso a un solo núcleo por defecto, y entonces los T hilos de un proceso se pelean ese núcleo y el híbrido no escala. En el i9, con 4 procesos, 1, 2 y 4 hilos tardaban casi lo mismo. Por eso el híbrido se lanza con `mpirun --bind-to none`, como lo hacen `verificar.sh` y `campana.sh` (Open MPI Project, s. f.). En macOS Open MPI no ata procesos, así que allá no cambia nada.

La llave la halla el hilo g = k mod pT, tras ⌊k/(pT)⌋ + 1 iteraciones. Entonces S_{iter} = (k + 1) / (⌊k/(pT)⌋ + 1), que ronda p·T y nunca lo supera. Con T = 1 es exactamente el intercalado, y para la eficiencia, el costo y la efectividad se usa w = p·T.

El híbrido usa todos los núcleos con pocos procesos MPI, lo que significa menos mensajes y menos memoria que `intercalado -np p·T`, y conserva un speedup consistente para cualquier llave. A cambio, agrega un nivel de sincronización y depende de cómo el lanzador ata los procesos. Si hay más trabajadores que núcleos de rendimiento, los hilos que caen en núcleos de eficiencia frenan al resto, porque el reparto es estático.

La consistencia con la versión sin OpenMP se demuestra con tres comparaciones de la campaña. Con T = 1, el híbrido debe tardar lo mismo que el intercalado. Con T = 2 y 4, el tiempo debe bajar de forma pareja en las tres llaves. Frente a `intercalado -np p·T`, con los mismos trabajadores pero todos MPI, el tiempo debe ser parecido.

Tabla teórica

La tabla da S_{iter} para cada llave con 56 bits, `-np 4` y C = 2^{16}.

Llave	Naive	Intercalado	Bloque-cíclico	Maestro-trabajador
a · 123456	1	4.00	2.13	2.13
b · 2^{54} − 1	1	4	4	3
c · 2^{54}	$1.8 \cdot 10^{16}$ (aparente)	4	4	3
fácil · 2^{55} + 1	$1.8 \cdot 10^{16}$ (aparente)	4	4	3
media · 2^{55} + 2^{53}	5 (aparente)	4	4	3
difícil · ⌈2^{56}/7 + 2^{56}/13⌉	1	4	4	3

Los valores del naive se sacan a mano con el bloque N/4 = 2^{54}. La llave media, k = N/2 + N/8, cae en el bloque del proceso 2, que empieza en N/2 y avanza N/8 + 1 llaves. Su speedup es (5N/8 + 1)/(N/8 + 1), es decir, 5.

La llave difícil vale cerca de 0.22·N, menos que N/4, así que cae en el bloque del proceso 0. Ese proceso hace las mismas k + 1 iteraciones que el secuencial y el speedup es 1. La llave c, N/4, es la primera del proceso 1, y la fácil, N/2 + 1, la segunda del proceso 2. En ambos casos el speedup ronda N/4, cerca de $1.8 \cdot 10^{16}$.

Con 28 bits, que son los de las mediciones, la tabla es la misma salvo c y fácil en el naive, que valen cerca de $6.7 \cdot 10^{7}$. Los presets escalan con N y cada llave conserva su posición relativa. Por eso medir con menos bits es representativo.

Consistencia frente a promedio

Si la llave fuera uniforme al azar entre 0 y N, el T_p esperado del naive y del intercalado sería el mismo. En ambos, el proceso que la tiene recorre en promedio la mitad de N/p llaves: $E[T_p] = N/(2p)$, mientras que $E[T_s] = N/2$. El speedup esperado es p en los dos, y el $S > p$ del naive con c, fácil y media se compensa con las llaves que le dan $S = 1$.

Lo que gana un reparto alternativo es que el speedup ya no depende de dónde cae la llave. Da cerca de p, o de $p - 1$, para cualquier llave, en vez de algo entre 1 y 10^{16}. Y si las llaves no son uniformes, por ejemplo si son pequeñas, el naive da $S = 1$ y el intercalado $S = p$.

Análisis de speedup y del comportamiento según la llave

Los números de este capítulo salen de la campaña en la laptop de Esteban (i9-13980HX), la máquina oficial del informe, con 28 bits, 5 repeticiones por caso y la mediana de cada caso. El detalle está en el Anexo 2 y en `resultados/analisis_escu.md`.

Las fórmulas son las del curso (Barlas, 2015; Pacheco, 2011). El speedup es $S = T_s / T_p$, la eficiencia $E = S / w$, el costo $C = w \cdot T_p$ y la efectividad $F = S / C$, con $w = p \cdot h$ trabajadores. T_s es siempre el programa `secuencial` con la misma llave y los mismos bits.

Lo teórico frente a lo medido

La tabla pone, para cada llave, el speedup por iteraciones que predice la teoría junto al speedup medido con `-np 4`.

Llave (28 bits)		Naive	Intercalado	Bloque-cíclico	Maestro-trabajador
fácil · k = 134 217 729	S_{iter} (teoría)	$6.7 \cdot 10^7$	4	4	3
	S medido	11 087 (aparente)	3.19	3.21	2.30
media · k = 167 772 160	S_{iter} (teoría)	5	4	4	3
	S medido	3.95	3.25	3.21	2.32
difícil · k = 58 996 804	S_{iter} (teoría)	1	4	4	3
	S medido	0.79	3.28	3.27	2.35

S_{iter}, que solo cuenta iteraciones, coincide exactamente con la teoría del capítulo de enfoques en todas las corridas. El reparto prueba las llaves que debe, y quien halla la llave lo hace en el número de intentos previsto. La diferencia entre S_{iter} y el S medido es, entonces, pura pérdida de velocidad por llave. La figura 11 muestra los speedups medidos.

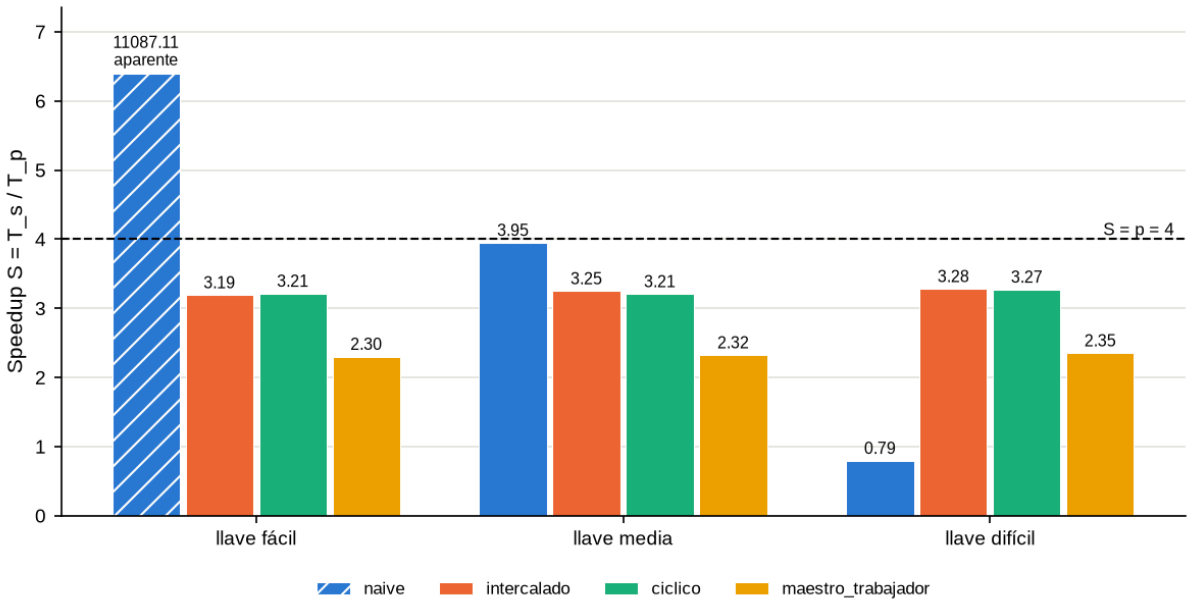


Figura 11. Speedup de cada programa por llave con 4 procesos. La barra del naive con la llave fácil está recortada.

Por qué el naive es inconsistente

El tiempo del naive no depende de cuántas llaves hay, sino de dónde cae la llave dentro del bloque del proceso que la tiene. Las tres llaves de la campaña lo muestran con los mismos 4 procesos.

La llave fácil, k = N/2 + 1, es la segunda del bloque del proceso 2. La halla en 2 intentos y 4 ms, que son casi todo arranque y la primera ronda de sondeo. El secuencial tarda 44.8 s, así que el speedup es cercano a 11 000. Con 56 bits sería del orden de 10^{16}.

La llave media, k = N/2 + N/8, cae a la mitad del bloque del proceso 2, que recorre N/8 llaves. La teoría da S_{iter} = 5, pero se midió S = 3.95. La razón es que los 4 procesos activos van cada uno al 79 % del ritmo del secuencial, y 5 × 0.79 ≈ 3.95.

La llave difícil vale cerca de 0.22·N y cae en el bloque del proceso 0. Ese proceso hace las mismas k + 1 iteraciones que el secuencial, así que S_{iter} = 1 y se midió S = 0.79. Es el peor caso: cuatro procesos tardan más que uno.

Entre llaves, el speedup del naive va de 0.79 a 11 000. Con las pruebas del enunciado pasa lo mismo: b y c son vecinas, y una da S ≈ 1 mientras la otra da S ≈ 10^{12}.

Por qué las alternativas son consistentes

El intercalado da speedups de 3.19 a 3.28 en las tres llaves, y el bloque-cíclico de 3.21 a 3.27. Es una variación menor al 3 %, frente a cuatro órdenes de magnitud en el naive.

El maestro-trabajador da de 2.30 a 2.35, también parejo, aunque por debajo de los otros dos porque el maestro no busca. Con 3 trabajadores su techo es 3, y respecto a ese techo su eficiencia es 2.32 / 3 = 0.77, igual a la de los otros.

En todos ellos, el que halla la llave hace $\lfloor k/p \rfloor + 1$ iteraciones o su equivalente por chunks. Por eso el speedup ronda p sin importar dónde esté la llave.

El superlineal es aparente

En esta campaña ningún programa alternativo superó S = w, y el análisis automático no levantó ninguna alerta. El único S mayor que p es el del naive con la llave fácil, y es aparente por tres razones.

La primera es que el naive no procesa llaves más rápido. El speedup de rendimiento compara llaves por segundo: S_{rend} = (llaves probadas / T_p) / ((k + 1) / T_s). Su máximo en toda la campaña es 3.37 con w = 4 y 10.05 con w = 16, así que S_{rend} nunca supera w. En el naive con la llave media vale 3.16: el proceso 2 solo tuvo que recorrer 1/8 del espacio, no lo recorrió más rápido.

La segunda es que se trata del efecto de búsqueda que se vio en el ejercicio 9 de la clase: la llave queda al inicio de un bloque que no es el primero. Las otras dos causas de superlinealidad no aplican. El cifrado mide 32 bytes y cabe en caché tanto en secuencial como en paralelo, y no hay sobrecarga que se reduzca al partir el problema.

La tercera es que en promedio el efecto desaparece. Si la llave es uniforme entre 0 y N, el proceso que la tiene recorre en promedio la mitad de N/p llaves. Entonces $E[T_p] = N/(2p)$ contra $E[T_s] = N/2$, y el speedup esperado del naive es p, igual que el del intercalado. Las llaves que le dan S mucho mayor que p se compensan con las que le dan S = 1.

Si un enfoque alternativo diera S > p, habría que revisar varias cosas en orden. Primero, que T_s sea de la misma llave y los mismos bits, y que las banderas de compilación coincidan. Después, que el secuencial no se haya medido con la máquina ocupada. Por último, que el reparto no se salte llaves, lo que se ve si S_{iter} difiere de la teoría, y que T_p no se mida después del aviso.

Fugas de eficiencia

Con `-np 4`, el intercalado tiene E ≈ 0.81, y además S_{rend} ≈ S (3.26 frente a 3.25). Eso significa que no se desperdician llaves: cada proceso prueba llaves más despacio que el secuencial solo. La tabla ubica cada fuga en nuestro problema.

Fuga	En nuestro problema
Parte secuencial (η_f)	Leer, cifrar y difundir el texto toma milisegundos. En el maestro-trabajador pesa el maestro entero, 1/p del cómputo
Comunicación (η_c)	Un `MPI_Test` cada 4096 llaves y un aviso al final: despreciable. En el maestro-trabajador, un pedido por chunk, oculto en parte por el pedido adelantado
Desbalance (η_b)	Nulo en los repartos estáticos, porque todos reciben N/p llaves. En el naive, respecto a la llave: p − 1 procesos trabajan donde no está
Redundancia (η_r)	Ninguna: cada llave se prueba una sola vez
Hardware	La que domina. Con un núcleo activo el i9 sube al turbo máximo; con 4 u 8 baja la frecuencia, y pasados 8 trabajadores se comparten núcleos por *hyperthreading* o entran los de eficiencia

Barrido de procesos, Amdahl y efectividad

La tabla resume el barrido con la llave media. La figura 12 lo compara con la recta ideal y con la curva de Amdahl.

Programa	p	T_p (s)	S	E	C (s)	F
secuencial	1	56.57	1.00	1.00	56.57	0.018
intercalado	2	32.57	1.74	0.87	65.13	0.027
intercalado	4	17.41	3.25	0.81	69.65	0.047
intercalado	8	11.30	5.01	0.63	90.40	0.055
maestro-trabajador	2	67.13	0.84	0.42	134.27	0.006
maestro-trabajador	4	24.36	2.32	0.58	97.45	0.024
maestro-trabajador	8	13.48	4.20	0.52	107.82	0.039

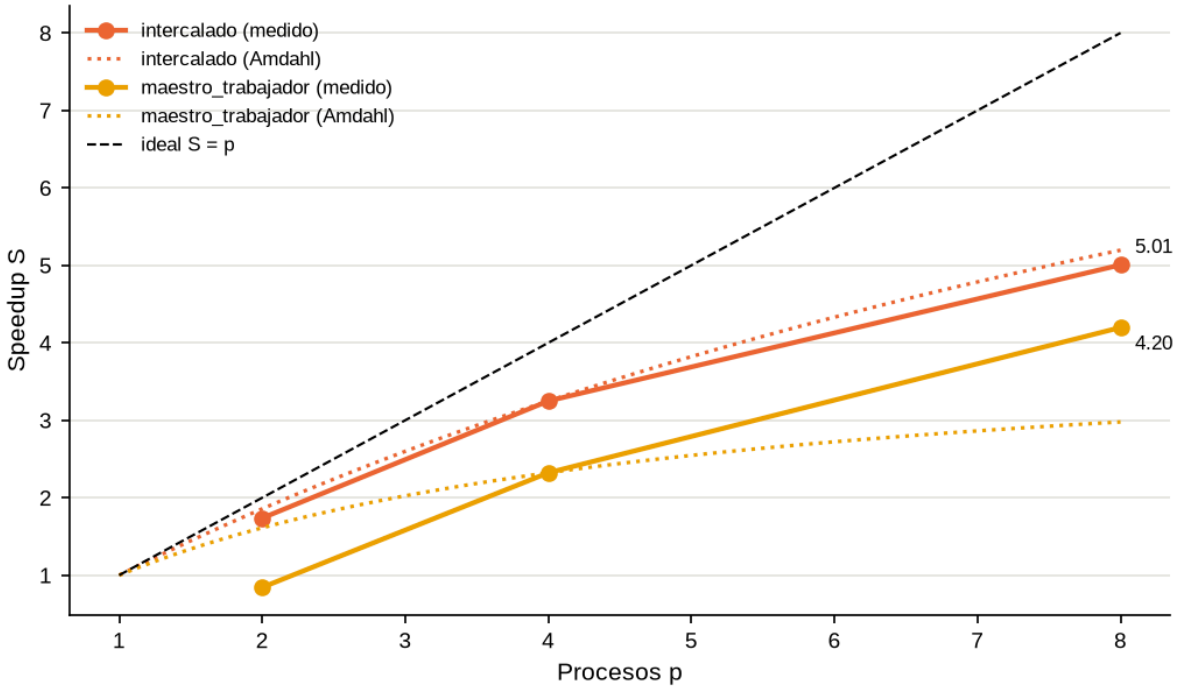


Figura 12. Speedup contra número de procesos con la llave media, con la recta ideal y la curva de Amdahl de cada programa.

Para Amdahl se toma el intercalado con p = 4. La fracción paralelizable es f_p = (1 − 1/3.25) / (1 − 1/4) = 0.923, así que f_s = 0.077 y el techo $1/f_s$ ronda 13. Con esa fracción, Amdahl predice 5.20 para p = 8 y se midieron 5.01, un 3.7 % menos. Para p = 2 predice 1.86 y se midieron 1.74.

Esa f_s no es código secuencial. Es la pérdida de frecuencia al activar más núcleos, que crece con p en vez de quedarse fija. Por eso Amdahl, calibrado en p = 4, sobrestima los extremos.

El maestro-trabajador no sigue a Amdahl, porque su pérdida es estructural: con p procesos tiene p − 1 trabajadores. Con p = 2 hay uno solo y es más lento que el secuencial (S = 0.84). Con p = 8 el maestro pesa 1/8 y el speedup sube a 4.20, muy por encima de los 2.98 que predice Amdahl con la f_s de p = 4. Su ideal es S = p − 1, y su eficiencia se acerca a 1 cuando p crece.

La ley de Gustafson describe mejor la fuerza bruta, porque el problema crece solo. Con S ≈ 3.25 se cubren 3.25 veces más llaves en el mismo tiempo, es decir, log2(3.25) ≈ 1.7 bits más. Con el híbrido de 16 trabajadores (S ≈ 10) son unos 3.3 bits más. Como f_s es casi 0, el speedup escalado de Gustafson tiende a p.

La efectividad sigue creciendo hasta el p más grande que se midió: 0.055 con 8 procesos en el intercalado y 0.105 con el híbrido de 16 trabajadores. Todavía conviene agregar

trabajadores, aunque la eficiencia baje de 0.87 a 0.63. Si se exige una eficiencia de al menos 0.8, el punto adecuado es p = 4.

El híbrido frente a la versión sin OpenMP

La tabla da el speedup y la eficiencia del híbrido con 4 procesos y 1, 2 y 4 hilos por proceso. La figura 13 muestra el tiempo y el speedup contra los hilos.

Llave	h = 1 (w = 4)	h = 2 (w = 8)	h = 4 (w = 16)
fácil	S = 3.32 · E = 0.83	S = 5.69 · E = 0.71	S = 11.08 · E = 0.69
media	S = 3.37 · E = 0.84	S = 5.75 · E = 0.72	S = 9.75 · E = 0.61
difícil	S = 3.35 · E = 0.84	S = 5.91 · E = 0.74	S = 9.92 · E = 0.62

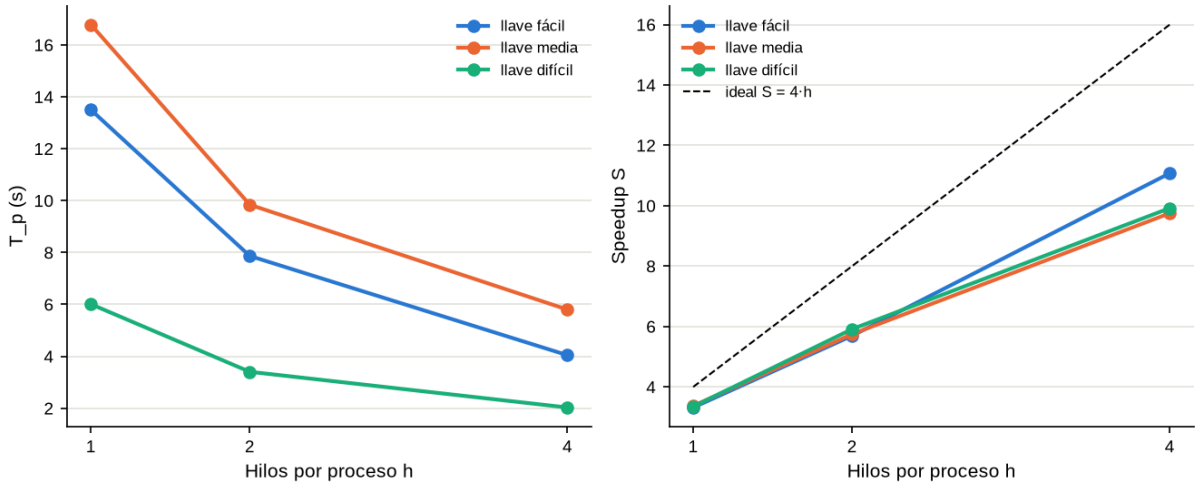


Figura 13. Tiempo y speedup del híbrido con 4 procesos según los hilos por proceso.

Con un hilo, el híbrido es el intercalado y tarda entre 0.96 y 0.98 veces lo mismo: 13.50 s frente a 14.04 s con la llave fácil, 16.78 frente a 17.41 con la media y 6.01 frente a 6.14 con la difícil.

Con más hilos, el tiempo baja de forma pareja en las tres llaves. El speedup ronda 5.7 a 5.9 con 2 hilos y 9.8 a 11.1 con 4. S_{iter} vale exactamente 8 y 16, y S_{rend} nunca supera w.

Frente a `intercalado -np 8`, que usa los mismos 8 trabajadores pero todos MPI, el híbrido de 4 procesos con 2 hilos tarda 9.84 s contra 11.30 s. Es un poco mejor porque los hilos comparten el proceso y no hay ocho procesos sondeando MPI.

La eficiencia baja con 16 trabajadores porque el i9 tiene 8 núcleos de rendimiento. Pasados 8 trabajadores, los demás comparten núcleo por *hyperthreading* o caen en núcleos de

eficiencia, que son más lentos. Como el reparto es estático, todos esperan al más lento. Aun así, el speedup crece de forma consistente con w.

De 28 a 56 bits

Al ritmo medido del secuencial, 167 772 161 llaves en 56.57 s, se prueban unos $2.97 \cdot 10^6$ llaves por segundo. Recorrer las 2^{56} llaves tomaría $2^{56} / 2.97 \cdot 10^6 \approx 2.43 \cdot 10^{10}$ s, unos 770 años.

El intercalado con 4 procesos (S = 3.25) bajaría ese tiempo a unos 237 años, y el híbrido de 16 trabajadores (S ≈ 9.75) a unos 79 años. Para recorrer el espacio completo en un día harían falta $2.43 \cdot 10^{10} / 86\,400 \approx 281\,000$ núcleos con eficiencia 1.

Como los presets escalan con N, S_{iter} con 28 bits es el mismo que con 56. Las únicas excepciones son las llaves c y fácil del naive, que valen cerca de N/4. Por eso estas mediciones representan el problema completo.

Conclusiones y recomendaciones

Conclusiones

El reparto decide el speedup, no el número de procesos. Con el mismo código de búsqueda y los mismos 4 procesos, el naive va de S = 0.79 con la llave difícil a S = 11 000 aparente con la fácil, que con 56 bits sería del orden de 10^{16}. En cambio, el intercalado y el bloque-cíclico dan de 3.19 a 3.28 con cualquier llave. El tiempo del naive depende de dónde cae la llave dentro del bloque de quien la tiene, y el de los repartos alternativos solo depende de k/p.

El proyecto no es superlineal. Los S mayores que p del naive son un efecto de búsqueda: el proceso que tiene la llave empieza cerca de ella, pero no prueba llaves más rápido. El speedup de rendimiento, en llaves por segundo, nunca pasó de p en ningún programa. Además, promediado sobre todas las llaves, el speedup esperado del naive es p, igual que el del intercalado. El análisis automático lo comprueba en cada campaña, y ningún enfoque alternativo dio S > w.

Lo que gana un enfoque alternativo es consistencia. El intercalado y el bloque-cíclico sostienen una eficiencia de 0.80 a 0.82 en las tres llaves. El maestro-trabajador queda en S ≈ 2.3 con 4 procesos, que es 0.77 de su techo de p − 1 = 3, porque el maestro no busca. A cambio, balancea solo la carga cuando los núcleos son desiguales.

La parte secuencial de Amdahl no es código secuencial. Leer y cifrar el texto toma milisegundos; la f_s que se despeja del speedup medido es pérdida efectiva. Ahí entran el turbo que baja con más núcleos activos, los núcleos de eficiencia, el sondeo de MPI y, en el maestro-trabajador, el propio maestro. Por eso Amdahl, calibrado con un valor de p, no predice bien los demás.

La fuerza bruta escala al estilo de Gustafson. El trabajo crece solo, porque cada bit duplica el espacio, y con w trabajadores se cubren log2(w) bits más en el mismo tiempo. Al ritmo medido de un núcleo, $2.97 \cdot 10^6$ llaves por segundo, recorrer las 2^{56} llaves tomaría unos 770 años, y hacerlo en un día exigiría unos 281 000 núcleos.

El híbrido MPI + OpenMP da un speedup consistente con la versión sin OpenMP. Con un hilo es el intercalado, con un tiempo de 0.96 a 0.98 del suyo. Con 2 y 4 hilos el speedup sube a cerca de 5.8 y de 10, de forma pareja en las tres llaves. Con 8 trabajadores queda incluso algo mejor que el intercalado puro, siempre que los procesos no queden atados a un núcleo.

Retos y cómo se resolvieron

La tabla resume los problemas que aparecieron durante el proyecto y cómo se resolvieron con Open MPI y las herramientas del sistema.

Reto	Solución
`rpc/des_crypt.h` ya no existe en glibc moderna	DES de OpenSSL (`DES_ecb_encrypt`), verificado contra `openssl enc -des-ecb`
Errores impresos por todos los procesos y procesos colgados	Solo el proceso 0 lee y valida; difunde un encabezado de estado con `MPI_Bcast` y todos salen limpios
El aviso de llave no llegaba mientras se calculaba	`MPI_Irecv` antes de buscar y `MPI_Test` cada 4096 llaves, que hace avanzar la recepción
La prueba b con 56 bits no termina (unos 150 años)	Límite `-t` con un centinela que el proceso 0 difunde al vencer, y la misma prueba con 28 bits
Medir con 56 bits es imposible	Presets que escalan con $N = 2^{bits}$: la llave conserva su posición relativa y S_{iter}
El maestro-trabajador se colgaba si los chunks se acababan antes de hallar la llave	Protocolo con dos invariantes: una respuesta por pedido y un único mensaje final por trabajador
El híbrido no escalaba con más hilos	Open MPI 5 ata cada proceso a un núcleo, así que se lanza con `--bind-to none`
Varios hilos con un MPI sin soporte multihilo completo	`MPI_THREAD_FUNNELED`: solo el hilo principal llama a MPI

Aprendizajes

Un speedup aislado no dice nada sin el caso que se midió. Hay que reportar la llave, los bits, el T_s del secuencial real y la mediana de varias corridas. El speedup de rendimiento y el speedup por iteraciones ayudan a separar lo que aporta el paralelismo de lo que aporta la suerte de dónde cae la llave.

Cuando la comunicación es mínima, los detalles del lanzador y del hardware pesan tanto como el algoritmo. El *binding* de `mpirun`, el turbo y la mezcla de núcleos de rendimiento y de eficiencia cambiaron los resultados más que el propio reparto.

Un protocolo de mensajes necesita invariantes explícitas. Los casos borde, como tener pocos chunks o hallar la llave al final, son justo los que no aparecen con los parámetros por defecto.

Recomendaciones

Para fuerza bruta con llaves de distribución desconocida conviene un reparto intercalado o bloque-cíclico. Ambos dan el speedup esperado para cualquier llave sin comunicación extra. Si los núcleos son desiguales o la máquina es compartida, el maestro-trabajador es mejor opción, siempre que haya suficientes procesos para que el maestro pese poco.

Al medir, conviene usar siempre el secuencial real con las mismas banderas de compilación, la máquina sin carga y al menos 5 repeticiones. Cualquier S > p debe tratarse como un error hasta explicarlo. En los programas híbridos hay que revisar el *binding* con `mpirun --report-bindings` y no pedir más hilos que núcleos de rendimiento.

Para atacar los 56 bits reales, el paso siguiente es un clúster o el uso de aceleradores como las GPU. El problema es vergonzosamente paralelo y escala con el número de núcleos.

Referencias

Amdahl, G. M. (1967). Validity of the single processor approach to achieving large scale computing capabilities. En *Proceedings of the AFIPS Spring Joint Computer Conference* (pp. 483–485). ACM. https://doi.org/10.1145/1465482.1465560

Barlas, G. (2015). *Multicore and GPU programming: An integrated approach*. Morgan Kaufmann.

Electronic Frontier Foundation. (1998). *Cracking DES: Secrets of encryption research, wiretap politics & chip design*. O'Reilly.

Fedora Project. (s. f.). *Changes/SunRPCRemoval: Removal of Sun RPC interfaces from glibc*. Fedora Project Wiki. https://fedoraproject.org/wiki/Changes/SunRPCRemoval

Gustafson, J. L. (1988). Reevaluating Amdahl's law. *Communications of the ACM, 31*(5), 532–533. https://doi.org/10.1145/42411.42415

ISO/IEC. (2011). *ISO/IEC 9899:2011. Information technology — Programming languages — C*.

Message Passing Interface Forum. (2023). *MPI: A message-passing interface standard (Version 4.1)*. https://www.mpi-forum.org/docs/

National Institute of Standards and Technology. (1999). *FIPS PUB 46-3: Data Encryption Standard (DES)*. U.S. Department of Commerce.

National Institute of Standards and Technology. (2005). Announcing approval of the withdrawal of Federal Information Processing Standard (FIPS) 46-3, Data Encryption Standard (DES). *Federal Register, 70*(96), 28907–28908.

Open MPI Project. (s. f.). *mpirun / prterun: Mapping, ranking, and binding* (Open MPI 5.0.x documentation). https://docs.open-mpi.org/

OpenMP Architecture Review Board. (2021). *OpenMP application programming interface (Version 5.2)*. https://www.openmp.org/specifications/

OpenSSL Project. (s. f.). *DES_random_key, DES_set_key, DES_ecb_encrypt: DES encryption* (OpenSSL 3 manual pages). https://docs.openssl.org/3.0/man3/DES_random_key/

Pacheco, P. (2011). *An introduction to parallel programming*. Morgan Kaufmann.

Stallings, W. (2020). *Cryptography and network security: Principles and practice* (8.ª ed.). Pearson.

Weimer, F. (2020, julio). *[PATCH] Remove --enable-obsolete-rpc configure flag* [Mensaje en lista de correo]. libc-alpha. https://inbox.sourceware.org/libc-alpha/20200711181508.GA9224@x230/t

Anexo 1: Catálogo de funciones y librerías

Todas las funciones del proyecto, incluidas las `static`, agrupadas por archivo. Cada una lleva sus entradas y salidas (nombre, tipo y uso) y una descripción. Al final están las funciones de librería que usamos: OpenSSL, Open MPI, la libc, OpenMP y los módulos de Python del análisis.

`des_utils.c`: DES sobre OpenSSL

`build_des_key`

Entradas	Tipo	Uso
`key56`	`uint64_t`	Llave numérica; solo se usan sus 56 bits bajos
`out`	`unsigned char[8]`	Buffer donde se escribe la llave DES

Salidas	Tipo	Uso
`out`	`unsigned char[8]`	8 bytes con 7 bits de llave y 1 de paridad cada uno

Descripción. Reparte los 56 bits en grupos de 7, uno por byte (bits 1..7), y completa la paridad impar con `DES_set_odd_parity`, igual que el `decrypt()` del `bruteforce.c` original.

`run_des` (`static`)

Entradas	Tipo	Uso
`key`	`uint64_t`	Llave de 56 bits
`buffer`	`unsigned char *`	Datos que se cifran o descifran en el mismo lugar
`length`	`size_t`	Bytes de `buffer`, múltiplo de 8
`direction`	`int`	`DES_ENCRYPT` o `DES_DECRYPT`

Salidas	Tipo	Uso
`buffer`	`unsigned char *`	Datos ya transformados

Descripción. Arma la llave, genera el *key schedule* con `DES_set_key_unchecked` (sin rechazar llaves débiles) y aplica `DES_ecb_encrypt` bloque a bloque.

`encrypt_des` y `decrypt_des`

Entradas	Tipo	Uso
`key`	`uint64_t`	Llave de 56 bits
`buffer`	`unsigned char *`	Texto plano (cifrar) o cifrado (descifrar)
`length`	`size_t`	Bytes de `buffer`, múltiplo de 8

Salidas	Tipo	Uso
`buffer`	`unsigned char *`	Resultado, en el mismo buffer

Descripción. Cifran o descifran con DES en modo ECB llamando a `run_des`. Reemplazan a `encrypt` y `decrypt` del `bruteforce.c`.

`try_key`

Entradas	Tipo	Uso
`key`	`uint64_t`	Llave candidata
`cipher`	`const unsigned char *`	Texto cifrado original (no se modifica)
`length`	`size_t`	Bytes del cifrado
`phrase`	`const char *`	Frase clave conocida
`workspace`	`unsigned char *`	Buffer de `length + 1` bytes reservado una sola vez

Salidas	Tipo	Uso
retorno	`int`	1 si el descifrado contiene la frase, 0 si no

Descripción. Copia el cifrado al `workspace` con `memcpy`, lo descifra con la llave candidata y busca la frase con `strstr`.

`padded_length`

Entradas	Tipo	Uso
`length`	`size_t`	Longitud original

Salidas	Tipo	Uso
retorno	`size_t`	Longitud redondeada al siguiente múltiplo de 8

Descripción. Calcula cuántos bytes ocupa el texto con relleno de ceros hasta completar bloques DES.

`options.c`: Parámetros

`print_usage` (`static`)

Entradas	Tipo	Uso
`program`	`const char *`	Nombre del programa
`flags`	`int`	Opciones extra que admite (`OPT_OUTPUT`, `OPT_CHUNK`)

Salidas	Tipo	Uso
(stderr)	texto	Ayuda del programa

Descripción. Imprime la forma de uso. `-c` y `-o` solo aparecen en los programas que las admiten.

`parse_u64` (`static`)

Entradas	Tipo	Uso
`text`	`const char *`	Número en decimal o en hexadecimal con `0x`; acepta una `L` final
`value`	`uint64_t *`	Destino del número

Salidas	Tipo	Uso
retorno	`int`	0 si la conversión fue completa, -1 si hay basura, signo o desbordamiento
`*value`	`uint64_t`	El número convertido

Descripción. Envuelve `strtoull` y rechaza todo lo que no sea un número completo.

`resolve_key`

Entradas	Tipo	Uso
`text`	`const char *`	Llave escrita en `-k`: número o preset
`key_bits`	`int`	Bits del espacio de búsqueda
`key_out`	`uint64_t *`	Destino de la llave

Salidas	Tipo	Uso
retorno	`int`	0 si es válida y cabe en 2^{bits}, -1 si no
`*key_out`	`uint64_t`	La llave resuelta

Descripción. Traduce los presets `a`, `b`, `c`, `facil`, `media` y `dificil` (escalados con N = 2^{bits}) o convierte el número, y valida que la llave sea menor que N.

`parse_options`

Entradas	Tipo	Uso
`argc`, `argv`	`int`, `char`	Argumentos del programa
`program`	`const char *`	Nombre para la ayuda
`flags`	`int`	Opciones extra admitidas (`OPT_OUTPUT`, `OPT_CHUNK`)
`opts`	`program_options_t *`	Destino de los parámetros

Salidas	Tipo	Uso
retorno	`int`	`OPTIONS_OK`, `OPTIONS_HELP` u `OPTIONS_ERROR`
`*opts`	`program_options_t`	Archivo, llave, frase, bits, límite de tiempo, chunk y salida

Descripción. Lee las opciones con `getopt` y las valida: rangos de `-b`, `-t` y `-c`, que `-c` sea menor que `-b`, que la frase no esté vacía y que la llave sea válida. Los mensajes de error salen en español.

`problem.c`: Problema y reporte

`read_text_file` (`static`)

Entradas	Tipo	Uso
`path`	`const char *`	Ruta del `.txt`
`length_out`	`size_t *`	Destino de la longitud

Salidas	Tipo	Uso
retorno	`char *`	Texto terminado en `'\0'` (liberar con `free`), o `NULL` si falla
`*length_out`	`size_t`	Bytes útiles, sin los saltos de línea finales

Descripción. Lee el archivo completo, hasta 64 KiB. Rechaza archivos vacíos, demasiado grandes o con bytes nulos.

`problem_prepare`

Entradas	Tipo	Uso
`opts`	`const program_options_t *`	Parámetros validados

Entradas	Tipo	Uso
`problem`	`crack_problem_t *`	Destino del problema

Salidas	Tipo	Uso
retorno	`int`	0 si todo salió bien, -1 si no
`*problem`	`crack_problem_t`	Texto, cifrado con relleno, frase y tamaño del espacio

Descripción. Carga el texto, verifica que contenga la frase clave y lo cifra con la llave privada.

`problem_release`

Entradas	Tipo	Uso
`problem`	`crack_problem_t *`	Problema, completo o a medio construir

Salidas	Tipo	Uso
`*problem`	`crack_problem_t`	Punteros liberados y en `NULL`

Descripción. Libera el texto, el cifrado y la frase. Se puede llamar más de una vez.

`problem_print_header`

Entradas	Tipo	Uso
`program`	`const char *`	Nombre del programa
`opts`	`const program_options_t *`	Parámetros
`problem`	`const crack_problem_t *`	Problema preparado
`processes`	`int`	Número de procesos

Salidas	Tipo	Uso
(stdout)	texto	Encabezado de la corrida

Descripción. Imprime el archivo, la frase, la llave y el tamaño del espacio antes de buscar.

`problem_print_result`

Entradas	Tipo	Uso
`program`	`const char *`	Nombre del programa
`opts`	`const program_options_t *`	Llave real y límite de tiempo
`problem`	`const crack_problem_t *`	Para descifrar y mostrar el texto

Entradas	Tipo	Uso
`result`	`const search_result_t *`	Resultado de la búsqueda

Salidas	Tipo	Uso
(stdout)	texto	Reporte y línea `CSV`, de 11 columnas

Descripción. Muestra la llave hallada, quién la halló y en cuántas iteraciones, el texto descifrado, las llaves probadas, el ritmo y el tiempo, y deja la línea CSV para la bitácora.

`print_hex`

Entradas	Tipo	Uso
`label`	`const char *`	Etiqueta de la línea
`bytes`	`const unsigned char *`	Datos
`length`	`size_t`	Bytes a imprimir

Salidas	Tipo	Uso
(stdout)	texto	Bytes en hexadecimal, en grupos de 8

Descripción. Imprime un buffer en hexadecimal, separado por bloques DES.

`wall_seconds`

Entradas	Tipo	Uso
—	—	—

Salidas	Tipo	Uso
retorno	`double`	Segundos del reloj monotónico

Descripción. Reloj de pared del secuencial (`clock_gettime(CLOCK_MONOTONIC)`).

`cifrar.c`: Parte B.1

`write_cipher_file` (`static`)

Entradas	Tipo	Uso
`path`	`const char *`	Archivo de salida
`bytes`	`const unsigned char *`	Texto cifrado

Entradas	Tipo	Uso
`length`	`size_t`	Bytes a escribir

Salidas	Tipo	Uso
retorno	`int`	0 si se escribió completo, -1 si no

Descripción. Guarda el cifrado en binario y revisa tanto `fwrite` como `fclose`.

`main` (cifrar)

Entradas	Tipo	Uso
`argc`, `argv`	`int`, `char`	`-f`, `-k`, `-p`, `-b`, `-o`

Salidas	Tipo	Uso
retorno	`int`	`EXIT_SUCCESS` si el descifrado coincide con el original
(stdout)	texto	Texto, cifrado en hexadecimal, descifrado y contraejemplo

Descripción. Cifra el `.txt` con la llave dada, lo descifra de vuelta, lo compara con el original y muestra qué sale con la llave vecina.

`secuencial.c`: Parte A.3

`main` (secuencial)

Entradas	Tipo	Uso
`argc`, `argv`	`int`, `char`	`-f`, `-k`, `-p`, `-b`, `-t`

Salidas	Tipo	Uso
retorno	`int`	`EXIT_SUCCESS` si la halló o si se usó `-t`
(stdout)	texto	Reporte y línea CSV

Descripción. Recorre las llaves 0, 1, 2, … hasta que `try_key` acierta, o hasta que vence el límite de tiempo, que revisa cada 4096 llaves.

`search_mpi.c`: Módulo común MPI

`validate_processes` (`static`)

Entradas	Tipo	Uso
`size`	`int`	Número de procesos
`min_processes`	`int`	Mínimo que necesita el programa
`problem`	`const crack_problem_t *`	Para comparar con 2^{bits}

Salidas	Tipo	Uso
retorno	`int`	0 si el número de procesos sirve, -1 si no

Descripción. El proceso 0 rechaza menos procesos que el mínimo (el maestro-trabajador pide 2) o más procesos que llaves.

`share_problem` (`static`)

Entradas	Tipo	Uso
`search`	`mpi_search_t *`	Búsqueda; en el proceso 0 ya preparada
`status`	`int`	Estado del proceso 0: listo, ayuda o error

Salidas	Tipo	Uso
retorno	`int`	El estado del proceso 0, igual en todos
`*search`	`mpi_search_t`	En los demás procesos: cifrado, frase, bits, límite y chunk

Descripción. Difunde con `MPI_Bcast` un encabezado de 6 enteros (estado, bits, longitudes, límite de tiempo y chunk) y luego el cifrado y la frase. Si el estado no es «listo», todos salen sin quedar esperando.

`mpi_search_setup`

Entradas	Tipo	Uso
`argc`, `argv`	`int *`, `char *`	Argumentos (se pasan a `MPI_Init`)
`program`	`const char *`	Nombre del programa
`option_flags`	`int`	0 u `OPT_CHUNK`
`min_processes`	`int`	Mínimo de procesos
`search`	`mpi_search_t *`	Destino del estado

Salidas	Tipo	Uso
retorno	`int`	`SEARCH_READY`, `SEARCH_HELP` o `SEARCH_ERROR`
`*search`	`mpi_search_t`	Rango, procesos, parámetros, problema y `workspace`

Descripción. Inicia MPI con `MPI_Init`, salvo que ya esté iniciado (`MPI_Initialized`; el híbrido lo inicia antes con `MPI_Init_thread`). El proceso 0 lee los parámetros y el archivo y valida el número de procesos; luego se reparte el problema, el proceso 0 imprime el encabezado y cada proceso reserva su `workspace`. Si no hay que buscar, cierra MPI antes de volver.

`mpi_search_release`

Entradas	Tipo	Uso
`search`	`mpi_search_t *`	Búsqueda

Salidas	Tipo	Uso
—	—	Memoria liberada y MPI cerrado

Descripción. Libera el `workspace` y el problema y llama a `MPI_Finalize`.

`mpi_search_begin`

Entradas	Tipo	Uso
—	—	—

Salidas	Tipo	Uso
retorno	`double`	`MPI_Wtime` justo después de la barrera

Descripción. Sincroniza a todos con `MPI_Barrier` y marca el inicio de la búsqueda.

`notify_all` (`static`)

Entradas	Tipo	Uso
`message`	`const uint64_t[3]`	{llave, proceso, iteraciones}
`size`	`int`	Número de procesos

Salidas	Tipo	Uso
—	—	Un `MPI_Send` a cada proceso

Descripción. Envía el aviso a todos, incluido uno mismo, para que todos los `MPI_Irecv` se completen.

`announce_timeout` (`static`)

Entradas	Tipo	Uso
`listener`	`found_listener_t *`	Antena del proceso 0
`size`	`int`	Número de procesos

Salidas	Tipo	Uso
`listener->timeout_sent`	`int`	Queda en 1

Descripción. Avisa a todos con la llave centinela `UINT64_MAX` que venció el límite de tiempo.

`found_listener_start`

Entradas	Tipo	Uso
`listener`	`found_listener_t *`	Destino de la antena
`search`	`const mpi_search_t *`	Para el límite de tiempo

Salidas	Tipo	Uso
`*listener`	`found_listener_t`	`MPI_Irecv` puesto, reloj iniciado y plazo (solo en el proceso 0)

Descripción. Pone la antena antes de buscar, sincroniza y arranca el reloj.

`found_listener_poll`

Entradas	Tipo	Uso
`listener`	`found_listener_t *`	Antena
`search`	`const mpi_search_t *`	Número de procesos

Salidas	Tipo	Uso
retorno	`int`	1 si ya llegó un aviso, 0 si no

Descripción. Revisa la antena con `MPI_Test`; el proceso 0 además manda el aviso de límite si el plazo ya venció.

found_listener_announce

Entradas	Tipo	Uso
search	const mpi_search_t *	Rango y número de procesos
key	uint64_t	Llave hallada
iterations	uint64_t	Llaves que probó este proceso

Salidas	Tipo	Uso
—	—	Aviso enviado a todos

Descripción. Arma {llave, rango, iteraciones} y lo envía con notify_all.

found_listener_try_key

Entradas	Tipo	Uso
listener	found_listener_t *	Antena
search	const mpi_search_t *	Cifrado, frase y workspace
key	uint64_t	Llave candidata
iterations	uint64_t *	Contador de llaves de este proceso

Salidas	Tipo	Uso
retorno	int	1 si hay que parar (la halló este proceso u otro avisó)
*iterations	uint64_t	Incrementado si se probó la llave

Descripción. Es un paso del bucle de los repartos estáticos: sondea cada 4096 llaves, prueba la llave y, si acierta, avisa a todos.

found_listener_finish

Entradas	Tipo	Uso
listener	found_listener_t *	Antena
search	const mpi_search_t *	Número de procesos
result	search_result_t *	Destino del resultado

Salidas	Tipo	Uso
*result	search_result_t	Hallazgo, llave, proceso, iteraciones y tiempo

Descripción. El proceso 0 sigue vigilando el plazo, con pausas de 1 ms, mientras espera. Luego todos cierran la antena con `MPI_Wait`, toman el tiempo y arman el resultado.

`mpi_search_report`

Entradas	Tipo	Uso
`search`	`const mpi_search_t *`	Búsqueda
`program`	`const char *`	Nombre del programa
`result`	`const search_result_t *`	Resultado (se usa en el proceso 0)
`own_iterations`	`uint64_t`	Llaves que probó este proceso

Salidas	Tipo	Uso
(stdout)	texto	Reporte y CSV, impresos por el proceso 0

Descripción. Suma las llaves de todos con `MPI_Reduce` y el proceso 0 llama a `problem_print_result`.

Programas MPI

`main` (naive, intercalado, ciclico)

Entradas	Tipo	Uso
`argc`, `argv`	`int`, `char`	`-f`, `-k`, `-p`, `-b`, `-t` (y `-c` en ciclico)

Salidas	Tipo	Uso
retorno	`int`	`EXIT_SUCCESS`, o `EXIT_FAILURE` si los parámetros son inválidos
(stdout)	texto	Reporte y CSV del proceso 0

Descripción. Los tres siguen el mismo esqueleto: `mpi_search_setup` → `found_listener_start` → bucle con `found_listener_try_key` → `found_listener_finish` → `mpi_search_report` → `mpi_search_release`. Solo cambia el recorrido. El naive prueba el bloque contiguo `[r·N/p, (r+1)·N/p)`, el intercalado prueba `r, r+p, r+2p, …` y el cíclico recorre los chunks `r, r+p, …` de 2^c llaves, cada uno en orden.

`run_master` (`static`, maestro_trabajador)

Entradas	Tipo	Uso
`search`	`const mpi_search_t *`	Espacio, chunk, límite y procesos

Entradas	Tipo	Uso
`result`	`search_result_t *`	Destino del resultado

Salidas	Tipo	Uso
`*result`	`search_result_t`	Hallazgo, proceso, iteraciones y T_p

Descripción. Atiende pedidos con `MPI_Recv(ANY_SOURCE)`. A cada pedido (ASK) responde exactamente un mensaje: un chunk (`TAG_WORK`) o `TAG_STOP`. Responde STOP desde que llega el hallazgo (y ahí toma T_p), cuando vence el plazo o cuando se acaban los chunks. Los mensajes finales (FOUND o DONE) no llevan respuesta y descuentan un trabajador activo; termina cuando recibió el mensaje final de todos.

`search_chunk` (`static`, maestro_trabajador)

Entradas	Tipo	Uso
`search`	`const mpi_search_t *`	Cifrado, frase y `workspace`
`work`	`const uint64_t[2]`	{primera llave, una más allá de la última}
`iterations`	`uint64_t *`	Contador de llaves del trabajador
`found_key`	`uint64_t *`	Destino de la llave

Salidas	Tipo	Uso
retorno	`int`	1 si la halló, 0 si no

Descripción. Prueba en orden todas las llaves de un chunk.

`run_worker` (`static`, maestro_trabajador)

Entradas	Tipo	Uso
`search`	`const mpi_search_t *`	Búsqueda

Salidas	Tipo	Uso
retorno	`uint64_t`	Llaves que probó el trabajador

Descripción. Pide un chunk y, apenas lo recibe, pide el siguiente por adelantado (`MPI_Send` + `MPI_Irecv`). Prueba el actual y espera el próximo con `MPI_Wait`. Si halla la llave, envía de inmediato el mensaje final FOUND y recibe (y descarta) la respuesta pendiente; si recibe STOP sin hallarla, envía el mensaje final DONE.

`main` (maestro_trabajador)

Entradas	Tipo	Uso
`argc`, `argv`	`int`, `char`	`-f`, `-k`, `-p`, `-b`, `-t`, `-c`

Salidas	Tipo	Uso
retorno	`int`	`EXIT_SUCCESS`, o `EXIT_FAILURE` con parámetros inválidos o `-np 1`
(stdout)	texto	Reporte y CSV del proceso 0

Descripción. Prepara la búsqueda con mínimo 2 procesos. El proceso 0 corre `run_master` y los demás `run_worker`; al final, `mpi_search_report` con 0 llaves aportadas por el maestro.

`hibrido.c`: MPI + OpenMP

`init_threaded_mpi` (`static`)

Entradas	Tipo	Uso
`argc`, `argv`	`int *`, `char *`	Argumentos (se pasan a `MPI_Init_thread`)

Salidas	Tipo	Uso
retorno	`int`	0 si MPI da al menos `MPI_THREAD_FUNNELED`, −1 si no (ya avisó y cerró MPI)

Descripción. Inicia MPI con `MPI_Init_thread` pidiendo `MPI_THREAD_FUNNELED` (solo el hilo principal llama a MPI) y valida el nivel que el MPI instalado concede en `provided`.

`share_threads` (`static`)

Entradas	Tipo	Uso
`search`	`const mpi_search_t *`	Rango, procesos y espacio de llaves

Salidas	Tipo	Uso
retorno	`int`	Hilos por proceso T, o −1 si no sirven

Descripción. El proceso 0 lee T con `omp_get_max_threads` (es decir, de `OMP_NUM_THREADS`), valida $1 \le T \le 256$ y que $p \cdot T$ no supere las llaves del espacio, y lo difunde con `MPI_Bcast` para que todos usen el mismo T aunque el entorno de algún proceso difiera.

`search_thread` (`static`)

Entradas	Tipo	Uso
`listener`	`found_listener_t *`	Antena del proceso; solo la toca el hilo 0
`search`	`const mpi_search_t *`	Problema, rango y procesos
`threads`	`int`	Hilos por proceso T
`shared`	`thread_shared_t *`	Bandera de parada y hallazgo del proceso

Salidas	Tipo	Uso
retorno	`uint64_t`	Llaves que probó este hilo
`*shared`	`thread_shared_t`	Llave e iteraciones si este hilo la halló; `stop` levantada

Descripción. Cuerpo de la región paralela. El hilo t del proceso r es el trabajador global g = r·T + t y prueba g, g + pT, g + 2pT, … con su propio `workspace`. Cada `POLL_INTERVAL` llaves revisa la bandera atómica `stop`; el hilo 0 además llama a `found_listener_poll` (`MPI_Test`) y levanta la bandera si llegó un aviso de otro proceso. El hilo que halla la llave la guarda en una sección `critical` y levanta la bandera.

`main` (hibrido)

Entradas	Tipo	Uso
`argc`, `argv`	`int`, `char`	`-f`, `-k`, `-p`, `-b`, `-t`
`OMP_NUM_THREADS`	variable de entorno	Hilos por proceso (la lee el proceso 0)

Salidas	Tipo	Uso
retorno	`int`	`EXIT_SUCCESS`, o `EXIT_FAILURE` con parámetros o hilos inválidos
(stdout)	texto	Reporte y CSV del proceso 0, con `hilos = T`

Descripción. Inicia MPI con hilos, prepara la búsqueda con `mpi_search_setup`, acuerda T y pone la antena. Abre la región paralela (`#pragma omp parallel` con `reduction(+)` de las llaves probadas) y, ya fuera de ella, el hilo principal hace el `found_listener_announce` si un hilo halló la llave (FUNNELED). Cierra con `found_listener_finish` y `mpi_search_report`.

`scripts/analizar.py`: Análisis de la campaña

Funciones de Python que convierten `resultados/campana_<host>_*.csv` en `resultados/analisis_<host>.md` y las gráficas de `docs/img/`.

Función	Entradas	Salidas	Descripción
`Group` (clase)	caso, programa, procesos, hilos, bits, llave real	objeto con listas de tiempos, iteraciones y llaves probadas	Repeticiones de un mismo caso; `time`, `finder_iterations` y `keys_tried` dan la mediana y `workers` = p·h
`case_rank`, `program_rank`	`str` caso o programa	`int`	Orden fijo de llaves y programas en tablas y gráficas
`host_from_paths`	`list[str]` rutas	`str` host	Saca el host del nombre `campana_<host>_<fecha>.csv`
`load_groups`	`list[str]` rutas	grupos válidos, todos los grupos, `int` filas	Lee los CSV, agrupa y aparta las corridas con `llave_hallada != llave_real`
`sequential_times`	grupos	`dict` (caso, bits) → T_s	Mediana del `secuencial` por llave y bits
`metrics`	`Group`, `float` T_s	`dict` S, E, C, F, S_{iter}, S_{rend}	Las fórmulas de clase con w = p·h
`amdahl_fp`	`float` S, `int` p	`float` f_p	$f_p = (1 - 1/S)/(1 - 1/p)$
`amdahl_speedup`	`float` f_p, `float` p	`float` S	$1/(f_s + f_p/p)$
`gustafson_speedup`	`float` f_s, `int` p	`float` S	$p - f_s(p - 1)$
`ideal_speedup`	`Group`	`int`	p·h, o p − 1 en el maestro-trabajador
`note`	`Group`, métricas, lista de alertas	`str` nota	Regla del superlineal: «aparente (búsqueda)» en el naive; `REVISAR` si otro programa da S > w o cualquiera da S_{rend} > w (tolerancia 2 %)
`fmt`	`float`, dígitos	`str`	Número legible, en notación científica si es enorme o diminuto
`time_cell`, `label`	`Group`	`str`	Celda «mediana (mín–máx)» y nombre del programa con sus hilos
`main_table`	líneas, grupos, T_s, alertas	líneas de Markdown	Tabla de T_p, S, E, C, F, S_{iter}, S_{rend} y nota

Función	Entradas	Salidas	Descripción
`amdahl_section`	líneas, grupos, T_s, p principal, llave del barrido	`dict` programa → f_p	f_p, f_s, techo, Gustafson y bits extra; predicción de Amdahl frente al barrido
`style_axes`	eje de matplotlib	—	Rejilla y ejes discretos
`plot_speedup_by_key`	matplotlib, grupos, T_s, p, ruta	PNG	Barras de S por programa y llave con la línea S = p; el naive aparente se recorta y se rotula
`plot_sweep`	matplotlib, grupos, T_s, f_p, llave, ruta	PNG	S contra p con la ideal y la curva de Amdahl
`plot_hybrid`	matplotlib, grupos, T_s, ruta	PNG	Tiempo y speedup del híbrido contra los hilos
`plot_all`	grupos, T_s, f_p, p, llave, host, carpeta	`list[str]` rutas	Genera las gráficas si hay matplotlib; si no, avisa y sigue
`analyze`	rutas, salida, carpeta de imágenes	`list[str]` alertas	Arma el `.md` completo e imprime las alertas
`parse_args`, `main`	línea de comandos	código de salida	1 si hubo alguna alerta `REVISAR`

Librerías

OpenSSL (`<openssl/des.h>`, `-lcrypto`)

Función o tipo	Uso en el proyecto
`DES_cblock`	Bloque o llave de 8 bytes
`DES_key_schedule`	Subllaves de las 16 rondas
`DES_set_odd_parity`	Completa la paridad impar de cada byte de la llave
`DES_set_key_unchecked`	Genera el *key schedule* sin rechazar llaves débiles
`DES_ecb_encrypt`	Cifra o descifra un bloque de 8 bytes en modo ECB

Open MPI (`<mpi.h>`)

Primitiva	Uso en el proyecto
`MPI_Init`, `MPI_Finalize`	Abren y cierran MPI (`mpi_search_setup` y `mpi_search_release`)
`MPI_Init_thread`	El híbrido inicia MPI pidiendo `MPI_THREAD_FUNNELED`
`MPI_Initialized`	`mpi_search_setup` no vuelve a iniciar MPI si el híbrido ya lo hizo
`MPI_Comm_rank`, `MPI_Comm_size`	Rango propio y número de procesos
`MPI_Bcast`	Reparte el encabezado, el cifrado y la frase desde el proceso 0
`MPI_Barrier`	Sincroniza el inicio de la búsqueda
`MPI_Wtime`	Reloj de pared para T_p y para el límite de tiempo
`MPI_Irecv`	Antena de aviso y pedido adelantado del maestro-trabajador
`MPI_Send`	Aviso de llave, pedidos y respuestas del maestro-trabajador
`MPI_Recv`	El maestro recibe pedidos; el trabajador recibe chunks y STOP
`MPI_Test`	Sondeo de la antena cada 4096 llaves (hace avanzar la recepción)
`MPI_Wait`	Cierra la antena y espera el chunk adelantado
`MPI_Reduce`	Suma de las llaves probadas por todos
`MPI_Abort`	Solo si falta memoria en un proceso

libc

Función	Uso en el proyecto
`memcpy`	Copia del cifrado al `workspace` en cada intento
`memset`, `memchr`	Inicializar estructuras; detectar bytes nulos en el `.txt`
`strstr`	Buscar la frase clave en el texto descifrado
`strlen`, `strcmp`, `strcat`, `strdup`	Frase, presets de llave, optstring y copia de la frase
`getopt`	Leer las opciones de la línea de comandos
`strtoull`	Convertir números de 64 bits (`parse_u64`)
`malloc`, `calloc`, `free`	Memoria del problema y del `workspace` (`calloc` deja el relleno en ceros)
`fopen`, `fread`, `fwrite`, `fclose`, `ferror`	Leer el `.txt` y guardar el cifrado

Función	Uso en el proyecto
`printf`, `fprintf`, `snprintf`, `fflush`, `perror`, `strerror`	Reportes y mensajes de error
`clock_gettime`	Reloj monotónico del secuencial
`nanosleep`	Pausa de 1 ms del proceso 0 mientras vigila el límite

OpenMP y C11 (híbrido)

Primitiva	Uso en el proyecto
`#pragma omp parallel num_threads(T) reduction(+ : iterations)`	Región paralela con T hilos; suma las llaves probadas de todos los hilos
`#pragma omp critical(found_key)`	Guarda llave e iteraciones del primer hilo que la halla
`omp_get_max_threads`	El proceso 0 lee T de `OMP_NUM_THREADS`
`omp_get_thread_num`	Número de hilo t, para el trabajador global $g = r \cdot T + t$
`atomic_int`, `atomic_init`, `atomic_store`, `atomic_load_explicit` (`<stdatomic.h>`)	Bandera de parada compartida entre los hilos de un proceso

Python (análisis)

Módulo	Uso
`csv`, `statistics.median`	Leer la campaña y tomar la mediana de las repeticiones
`math.log2`	Bits extra según Gustafson
`argparse`, `os`, `re`, `sys`	Línea de comandos, rutas y alertas
`matplotlib` (opcional)	Gráficas en `docs/img/`

Anexo 2: Bitácora de pruebas y speedups

Máquina y software

Todas las mediciones de este anexo, la Parte A.3 y el análisis de speedup se tomaron en la laptop de Esteban, que es la máquina oficial del informe. Las pruebas B.1 y B.2 se hicieron en la MacBook Pro de Nico, como se indica en su capítulo. La tabla describe la máquina oficial.

Componente	Detalle
CPU	Intel Core i9-13980HX: 24 núcleos (8 de rendimiento con *hyperthreading* y 16 de eficiencia), 32 hilos
Memoria	16 GB
Sistema	CachyOS (Linux 7.2.9)
Compilador	GCC 16.2.1 con `-O2 -march=native -std=c11`, las mismas banderas en todos los programas
MPI y librerías	Open MPI 5.0.11, OpenSSL 3.6.5 y el OpenMP de GCC

La ficha completa, con la salida de `lscpu` y la versión de `mpirun`, queda en `resultados/maquina_escu.txt`.

Procedimiento

La campaña se corrió con los comandos siguientes, en este orden.

```
make clean && make
scripts/verificar.sh                                  # 126 casos, 0 fallas
OUT=resultados/secuencial_longitudes_escu.csv REPS=5 scripts/medir_longitudes.sh
# Parte A.3
BITS=28 REPS=5 NP=4 NP_LIST="2 4 8" HYBRID_THREADS="1 2 4" scripts/campana.sh
python3 scripts/analizar.py resultados/campana_escu_<fecha>.csv
python3 scripts/capturas.py                           # capturas de este anexo
```

Se usaron 28 bits, es decir, $N = 2^{28} = 268\,435\,456$ llaves. Los presets escalan con N, así que cada llave conserva la posición relativa que tiene con 56 bits. La llave fácil vale $N/2 + 1 = 134\,217\,729$, la media $N/2 + N/8 = 167\,772\,160$ y la difícil $\lceil N/7 + N/13 \rceil = 58\,996\,804$.

Cada caso se repitió 5 veces y se reporta la mediana, junto con el mínimo y el máximo. El tiempo es de pared. En los programas MPI se mide con `MPI_Wtime` desde la barrera hasta que llega el aviso, y en el secuencial con `clock_gettime(CLOCK_MONOTONIC)` alrededor de la búsqueda. T_s es la mediana del programa `secuencial` con la misma llave y los mismos bits,

no el paralelo con un proceso. El híbrido se lanzó con `mpirun --bind-to none -x OMP_NUM_THREADS -np 4` y 1, 2 y 4 hilos.

Verificación

El script `scripts/verificar.sh` usa `-b 20` y corre cada programa con las 6 llaves (a, b, c, fácil, media y difícil) y con 2, 3 y 4 procesos. El híbrido se prueba además con 1 y 2 hilos. El maestro-trabajador se prueba con pocos chunks (`-b 18`), que se agotan antes de hallar la llave, y con un solo proceso, donde debe dar error sin colgarse. El resultado fue de 126 casos y 0 fallas, como muestra la figura 14.

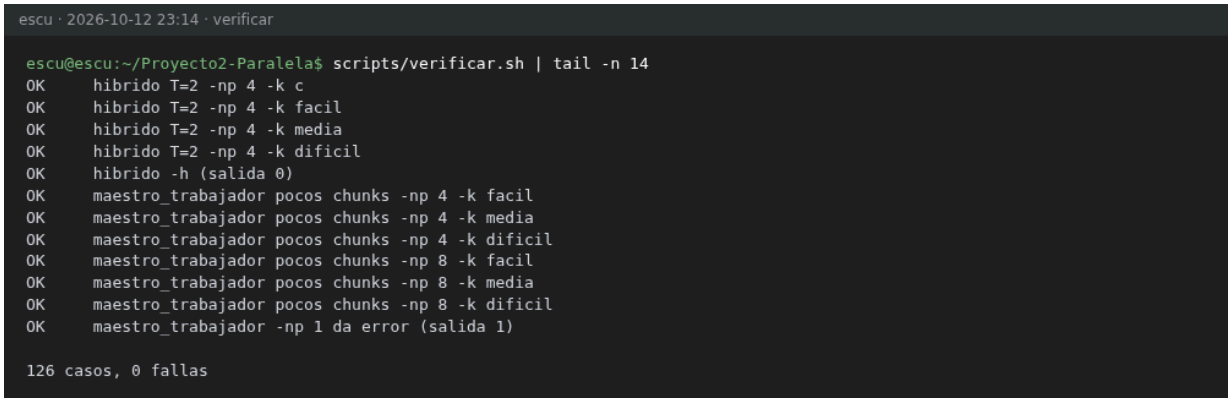


Figura 14. Final de la salida de `scripts/verificar.sh`, con los últimos casos y el resumen.

Pruebas B.1 y B.2

Las corridas del cifrado con la llave 42 y las pruebas a, b y c del naive con `-np 4` están en el capítulo de pruebas B, con sus capturas y el archivo `resultados/pruebas_b2.csv`. La prueba a da S ≈ 0.9 como mediana de 5 corridas. La b no termina con 56 bits, pues tomaría unos 149 años en secuencial, y con 28 bits da S = 0.96. La c la halla el proceso 1 al primer intento, con un S aparente de unos $3.9 \cdot 10^{12}$ con 56 bits.

Campaña: tiempos (mediana de 5, en segundos)

La campaña tuvo 140 corridas y las 140 hallaron la llave real. El mínimo y el máximo de cada caso están en `resultados/analisis_escu.md`.

Programa	p	h	fácil	media	difícil
secuencial (T_s)	1	1	44.792	56.569	20.127
naive	4	1	0.004	14.326	25.321
intercalado	4	1	14.040	17.414	6.136

Programa	p	h	fácil	media	difícil
ciclico (C = 2^{16})	4	1	13.942	17.645	6.152
maestro_trabajador (C = 2^{16})	4	1	19.472	24.362	8.563
hibrido	4	1	13.499	16.783	6.009
hibrido	4	2	7.868	9.842	3.404
hibrido	4	4	4.044	5.801	2.029

Speedups ($S = T_s / T_p$)

Programa	w	fácil	media	difícil	E (media)
naive	4	11 087 (aparente)	3.95	0.79	0.99
intercalado	4	3.19	3.25	3.28	0.81
ciclico	4	3.21	3.21	3.27	0.80
maestro_trabajador	4	2.30	2.32	2.35	0.58
hibrido, 1 hilo	4	3.32	3.37	3.35	0.84
hibrido, 2 hilos	8	5.69	5.75	5.91	0.72
hibrido, 4 hilos	16	11.08	9.75	9.92	0.61

El análisis no levantó ninguna alerta. Ningún enfoque alternativo da S > w y ningún programa da S_{rend} > w.

Barrido de procesos (llave media)

Programa	p = 2	p = 4	p = 8
intercalado: T_p (s) / S	32.565 / 1.74	17.414 / 3.25	11.301 / 5.01
maestro_trabajador: T_p (s) / S	67.133 / 0.84	24.362 / 2.32	13.477 / 4.20

Capturas

Cada captura muestra la salida real de los comandos, tal como la imprimen los programas. Se tomaron con `scripts/capturas.py` en la misma máquina, después de la campaña. Los tiempos de una corrida suelta varían unas décimas respecto a la mediana.

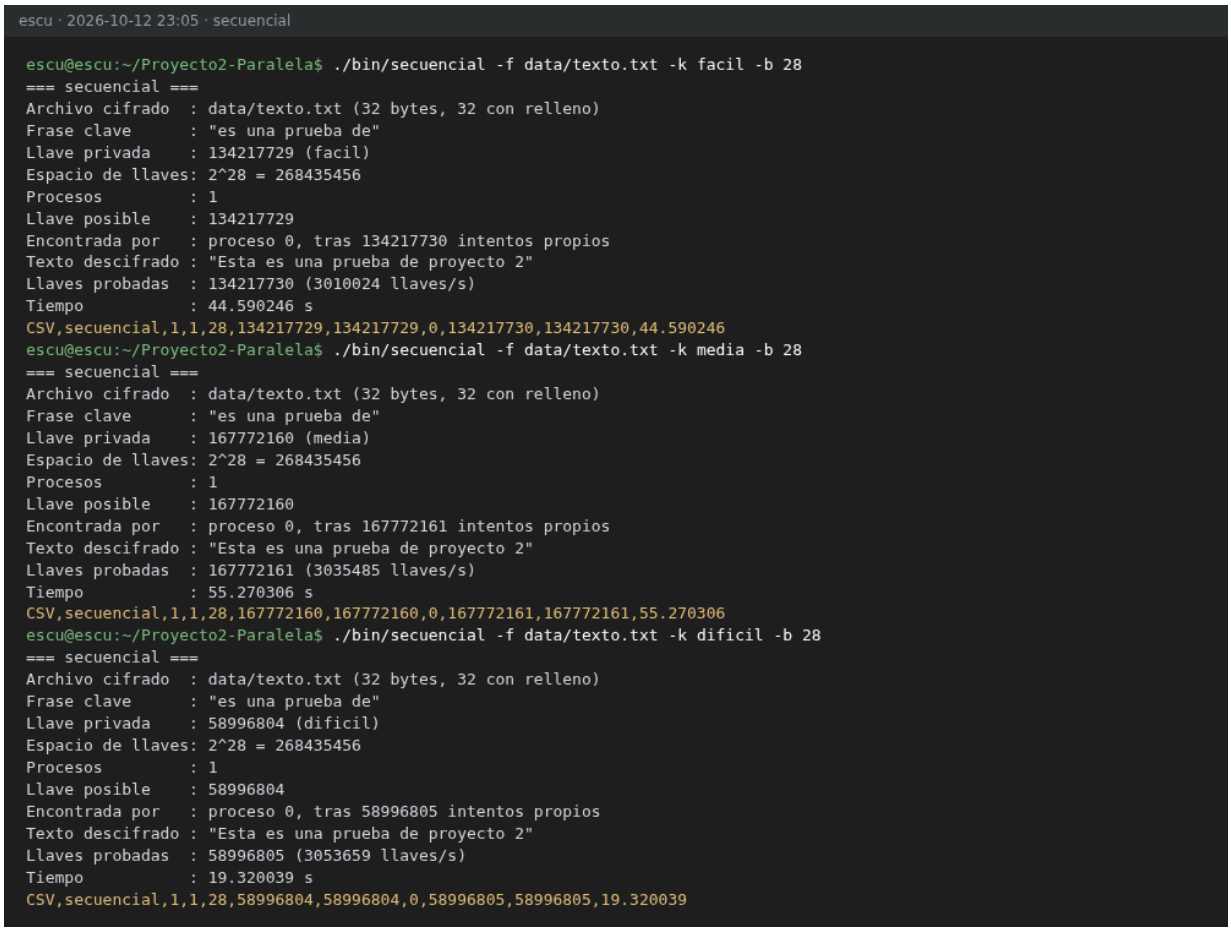


Figura 15. Secuencial con las llaves fácil, media y difícil; estas corridas dan el T_s.

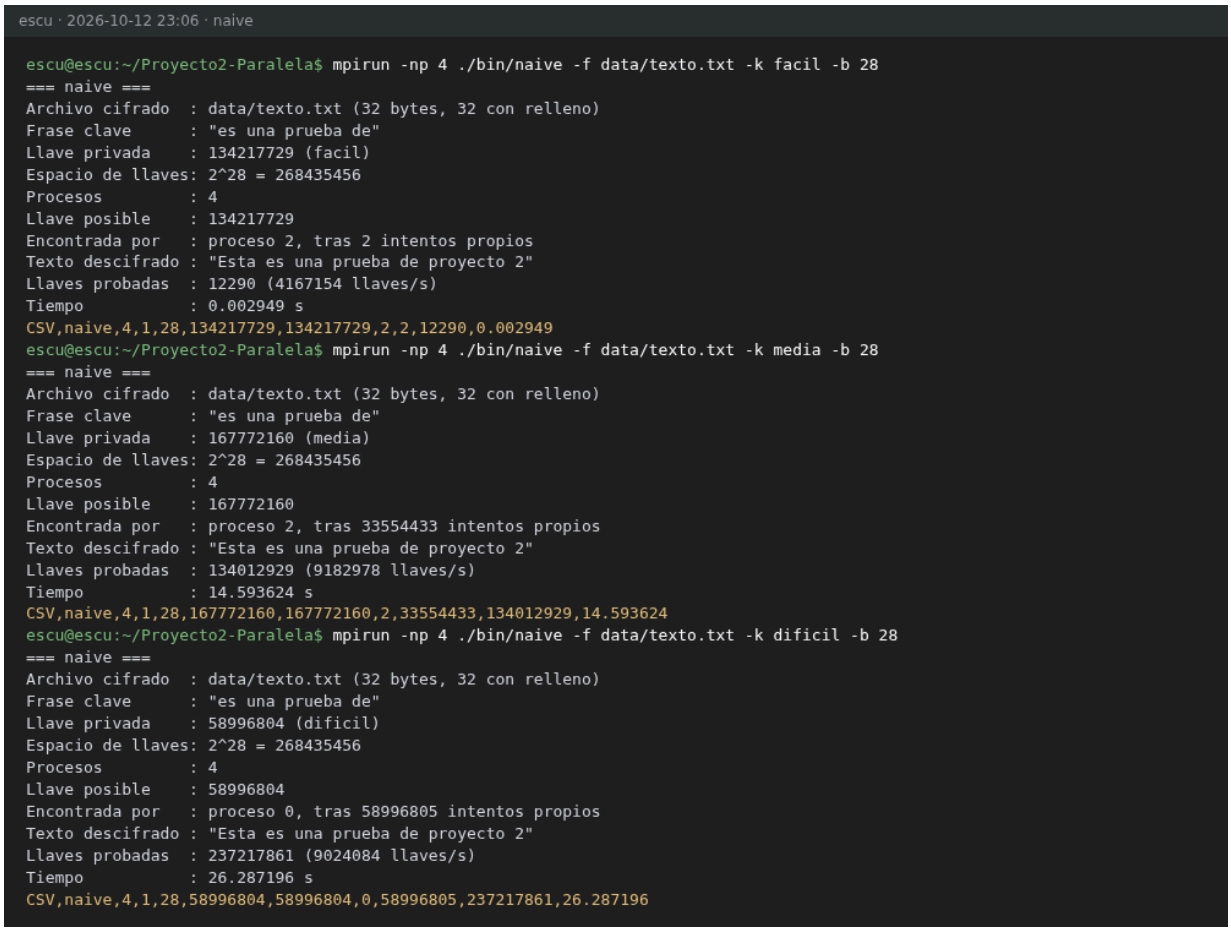


Figura 16. Naive con 4 procesos. Con la llave fácil, el proceso 2 la halla al segundo intento; con la media, a la mitad de su bloque; con la difícil, el proceso 0 la halla con las mismas iteraciones que el secuencial.

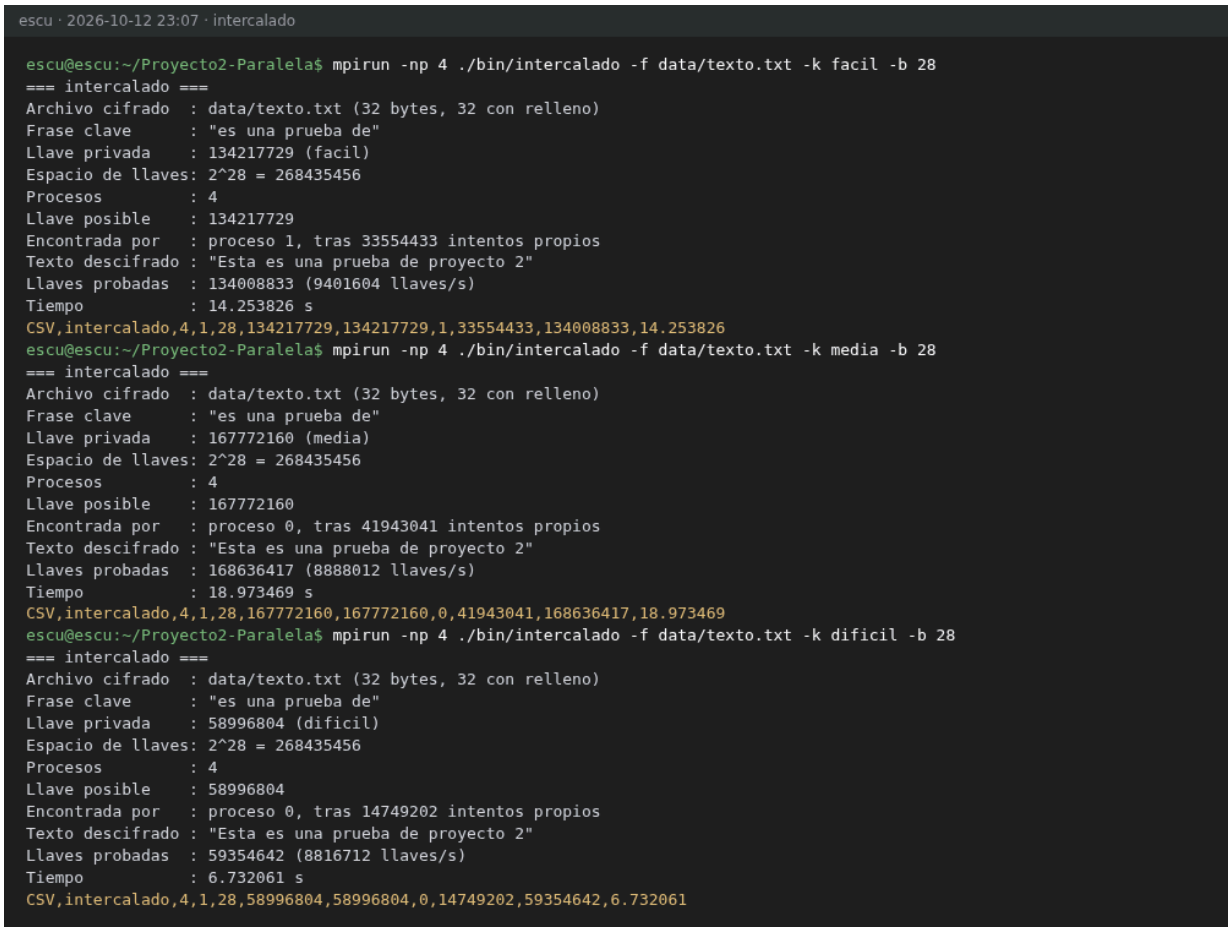


Figura 17. Intercalado con 4 procesos y las tres llaves.

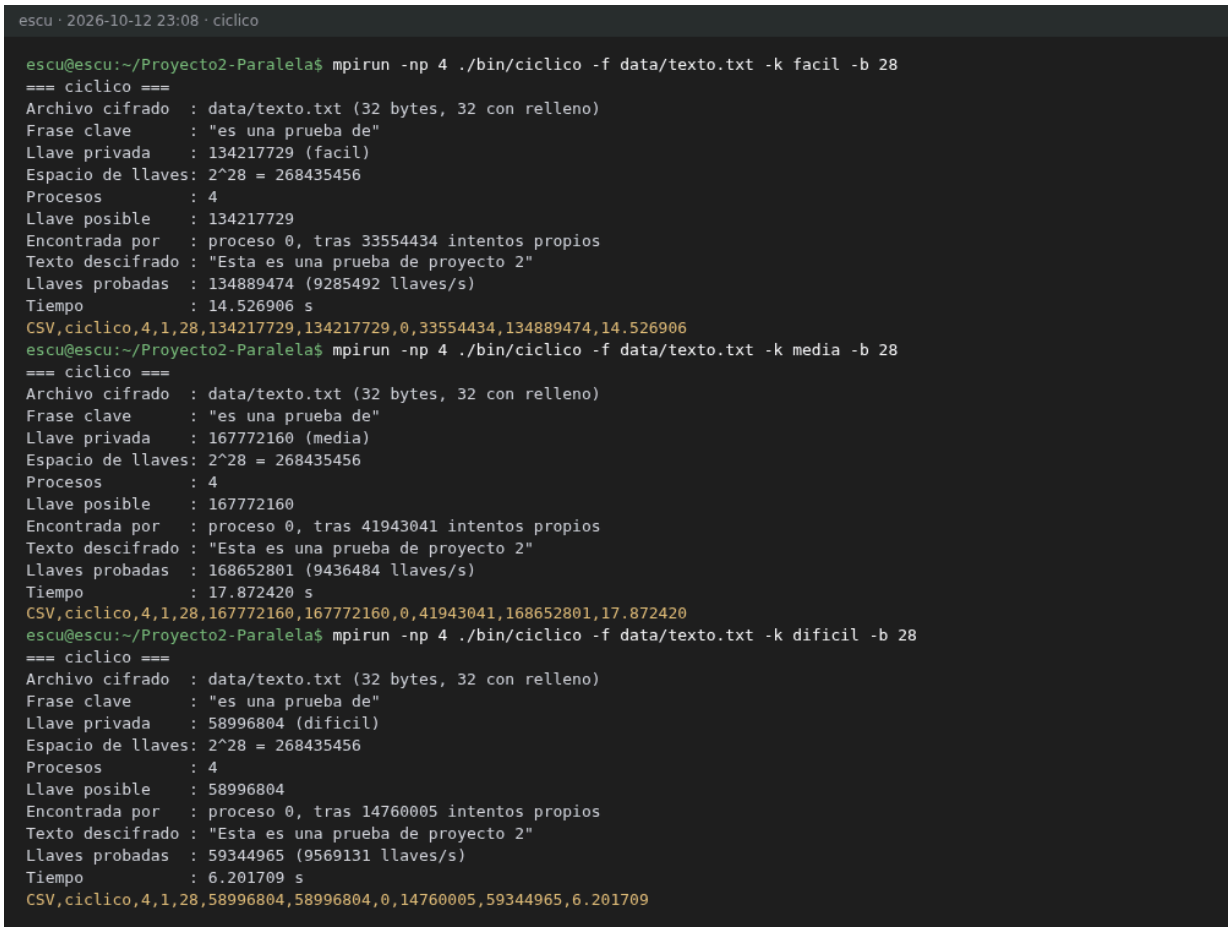


Figura 18. Bloque-cíclico con 4 procesos y chunks de 2^{16} llaves.

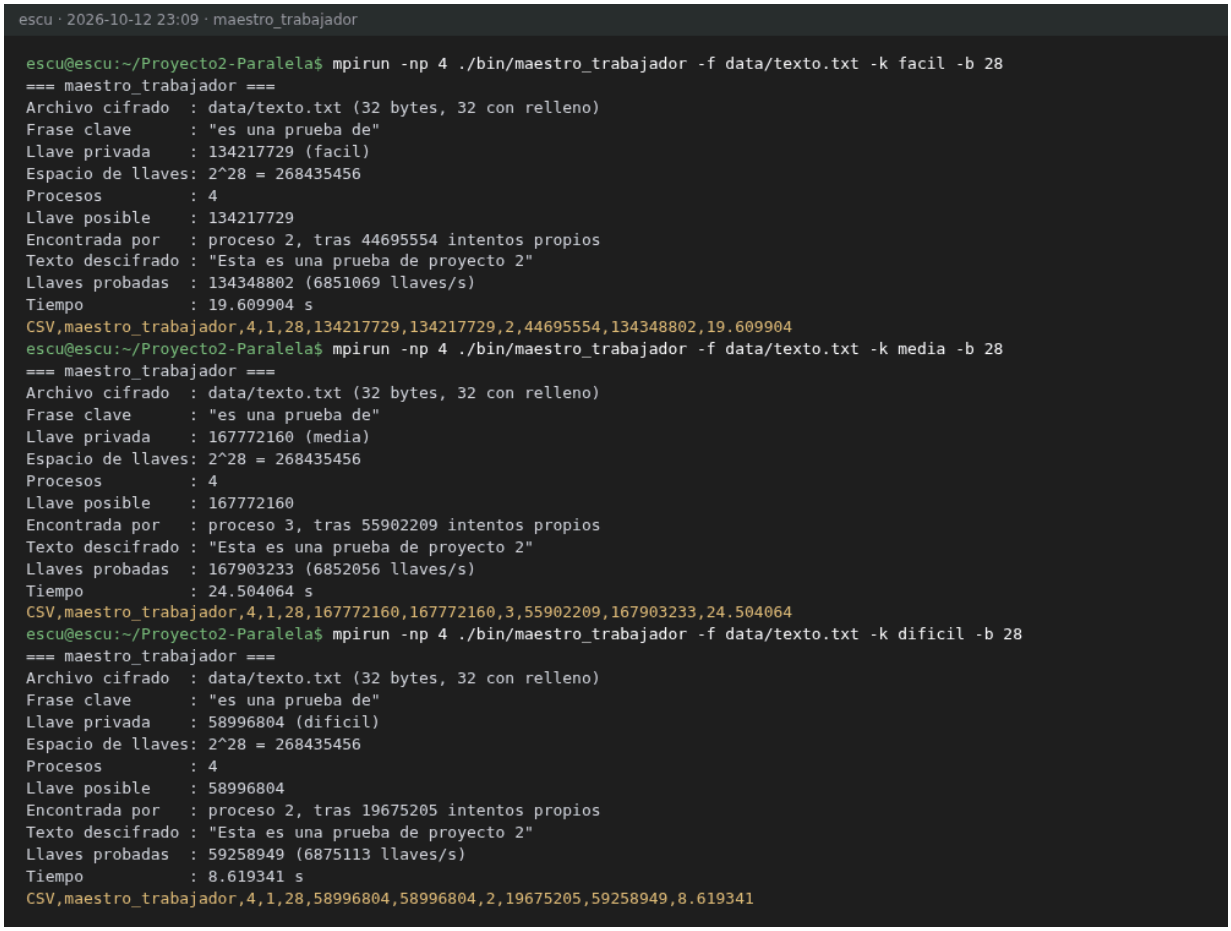


Figura 19. Maestro-trabajador con 4 procesos, es decir, un maestro y tres trabajadores.

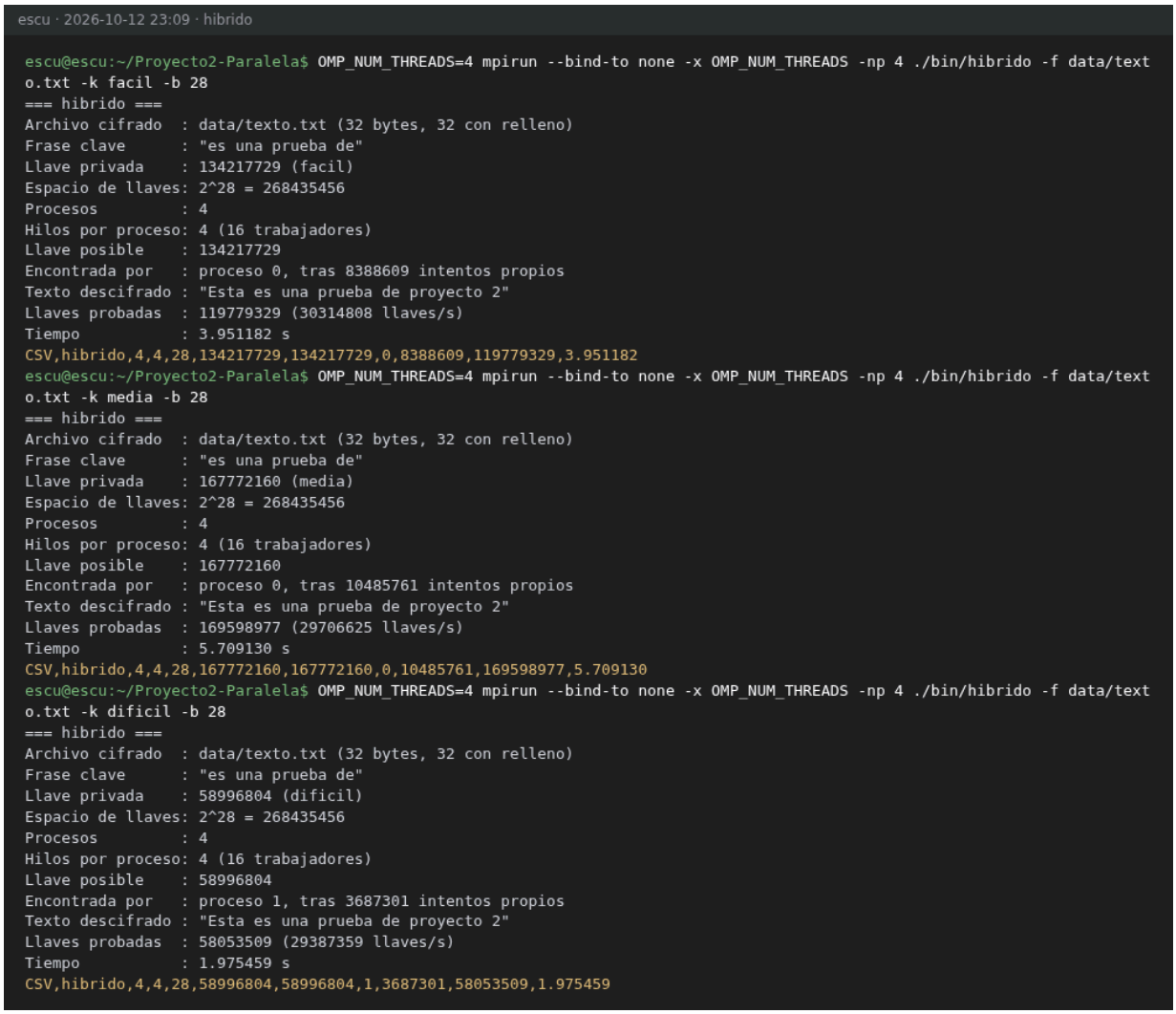


Figura 20. Híbrido con 4 procesos y 4 hilos por proceso.

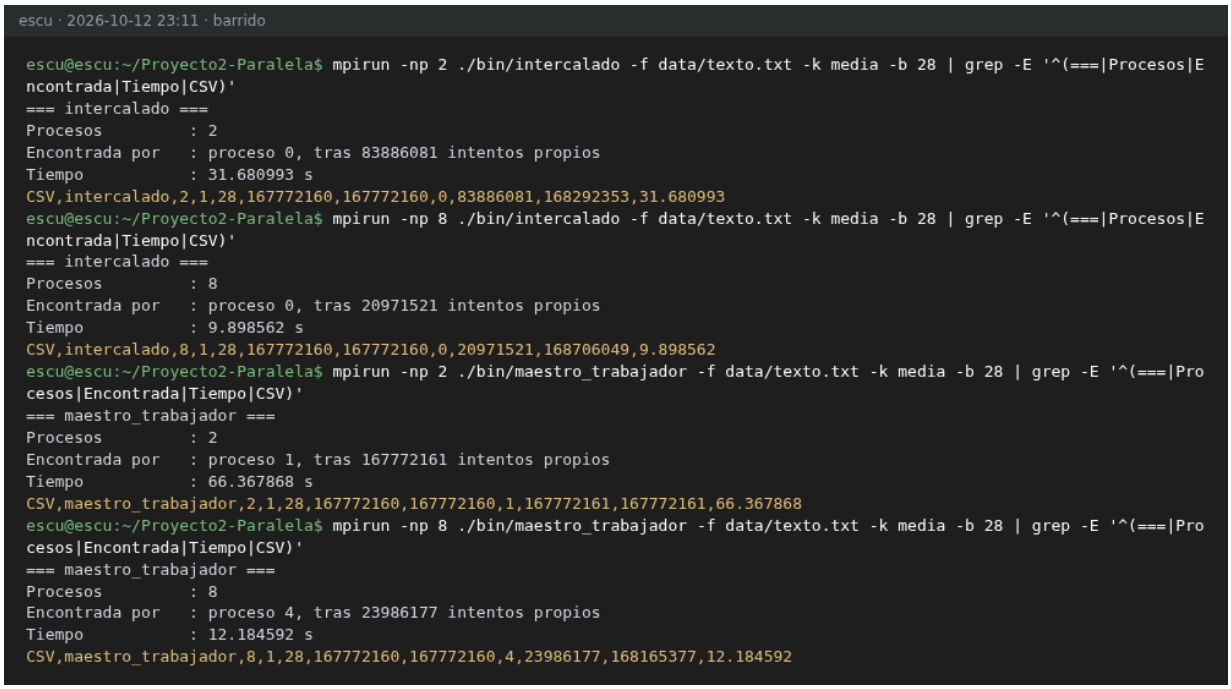


Figura 21. Barrido de procesos: intercalado y maestro-trabajador con 2 y 8 procesos. Solo se muestran las líneas clave de cada reporte.